About this course

- **The course is based on the latest stable versions of Playwright and Node.js at the time of delivering the training.**
- **Playwright releases new versions frequently, but the course covers fundamentals that apply to future versions.**
- **Course Details:**
  - Covers fundamental concepts that remain relevant for future Playwright updates.
  - Ensures practical, hands-on learning for effective test automation.

RUBRiC

Introduction to Playwright

1. Introduction to Playwright

- In this lesson you will learn about:
  - 1. 1 What is Playwright?
  - 1.1.1 Why Choose Playwright?
  - 1.1.2 Difference between Selenium and Playwright
  - 1.1.3 Features of Playwright
  - 1.1.4 Which one is preferred – Playwright or Selenium?
  - 1.1.5 Typescript fundamentals

RUBRiC

4

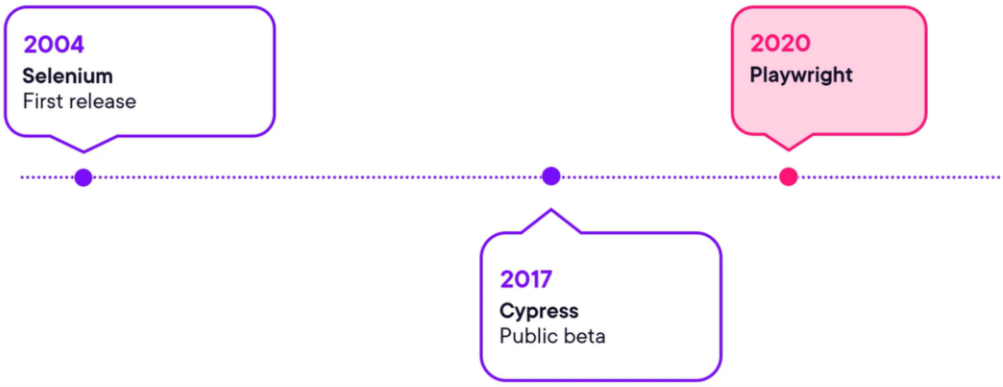
1.1 What is Playwright?

- **Playwright is a modern solution for writing robust end-to-end tests for web applications.**
- **Combines speed, stability, and usability with powerful built-in features.**
- **Actively maintained and continuously evolving with industry needs.**
- **Supports**

RUBRIC

- Playwright is an open-source NodeJS framework for Web Testing and Automation.
- It supports all modern rendering engines including Chromium, WebKit, and Firefox.
- Users can test on Windows, Linux, and macOS, locally or on CI, headless or headed.
- It offers end-to-end testing through its high-level API that allows the tester to control headless browsers.

1.1 When Playwright became widely used?



RUBRiC

1.1.1 Why Choose Playwright?

- Solid built-in auto-wait and stabilization mechanisms to greatly reduce flakiness.
- Native support for test isolation and parallel execution, check, built-in failure retries or screenshots on failures.
- Possibility to write your test in Java, C#, Python, JavaScript, or TypeScript with Node.js.

RUBRiC

Comparison of Playwright with other test automation tools

Feature	Selenium	Cypress	Playwright
Supported Languages	C#, Java, JavaScript, Ruby, Python	JavaScript	C#, Java, JavaScript, Python
Performance & Reliability	Can be slow/flaky if waiting is poor	Visual execution but can be slow	Typically the fastest execution
Browser Support	All major full browsers	Chrome, Edge, Firefox, Electron	Chromium+, Firefox, WebKit
Protocol Used	WebDriver protocol	In-browser JavaScript	Debug protocols
Licensing & Governance	Open source, standards, & gov	Open source from Cypress	Open source from Microsoft

Comparison of Selenium, Cypress, and Playwright

1.Supported Languages:

1. **Selenium** supports multiple programming languages, including **C#, Java, JavaScript, Ruby, and Python**, making it highly flexible.
2. **Cypress** only supports **JavaScript**, which limits its adoption in teams using other languages.
3. **Playwright** supports **C#, Java, JavaScript, and Python**, making it more versatile than Cypress.

2.Performance & Reliability:

1. **Selenium** can be **slow and flaky**, especially if proper waiting mechanisms aren't used.
2. **Cypress** runs tests visually but can still be **slow** due to its execution method.
3. **Playwright** is **typically the fastest** in execution due to its efficient architecture.

3.Browser Support:

1. **Selenium** supports **all major full browsers**, making it the most universal.
2. **Cypress** supports **Chrome, Edge, Firefox, and Electron**, but lacks full browser support.
3. **Playwright** supports **Chromium+, Firefox, and WebKit**, giving it better modern browser coverage.

4.Protocol Used:

1. **Selenium** relies on the **WebDriver protocol**, which communicates with browsers externally.
2. **Cypress** executes tests using **in-browser JavaScript**, which simplifies debugging but limits flexibility.
3. **Playwright** uses **debug protocols**, making it more efficient than WebDriver-based automation.

5.Licensing & Governance:

1. **Selenium** is **open-source** and maintained by a broad community with industry standards.
2. **Cypress** is also **open-source** but governed by Cypress itself.
3. **Playwright** is **open-source from Microsoft**, ensuring active development and support.

Key Takeaways:

•**Selenium** is the best for broad **language and browser support** but can be slow.

•**Cypress** is best suited for **JavaScript-based** teams with simpler testing needs.

•**Playwright** provides **high performance and modern browser support**, making it a strong alternative to Selenium.

Why Playwright?

Below are the advantages of using the test automation framework

- Dependable
- Efficient
- Capable
- Ubiquitous
- Delightful
- Lively

9

Why Playwright is a Reliable Choice?

- **Fast Issue Resolution**
- A dedicated development team triages issues within 2-3 days.
- Bugs are quickly fixed, and new features are implemented efficiently.

- Frequent & Rapid Releases
- Monthly (or near-monthly) releases, ensuring continuous improvements.
- Avoids the long development cycles of yearly updates.

- **Expertise Across the Stack**

•Developed by a **single team of browser engineering experts**.

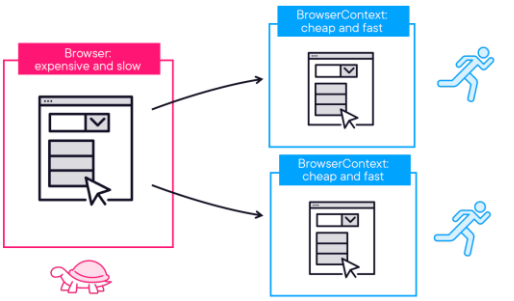
•Capable of working across:

✅ **Browsers** (Chromium, Firefox, WebKit)
✅ **Drivers**
✅ **Test automation framework**

Why Playwright ?

Efficient Test Execution with Playwright

- **Introduces browser contexts, acting as virtual containers for isolating:**
  - ✅ Cookies
  - ✅ Storage
  - ✅ Browsing history
- **Faster setup and teardown compared to restarting the browser.**
  - **Optimized Parallel Execution**
    - Built-in support for parallel test execution.
    - Enhances test efficiency without compromising stability.
    - Reduces overall test execution time significantly.



11

Note: **Browser**: A single instance of a web browser (e.g., Chromium, Firefox, WebKit) running in Playwright.It **launches** the actual browser process.

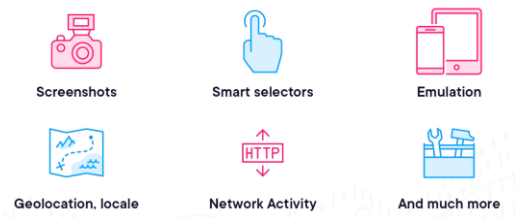
•Represents a **complete browser environment** where multiple pages or tabs can be opened.

•**Browser Context**: A **virtual container** inside a browser instance that allows the simulation of isolated sessions.It **does not launch a new browser** but instead simulates different user environments within the same browser instance.

Why Playwright ?

Capable: Playwright's Superior Testability and Built-in Auto-Waiting Mechanism

- **Auto-Waiting Mechanism**
  - Eliminates common UI automation challenges:
    - Issues like NoSuchElementException and ElementsNotVisibleException.
    - Occur when interacting with temporarily unavailable or disabled elements.
  - Traditional Automation Challenges:
    - Testers often had to:Implement additional checks to verify element visibility and interactivity.
    - Use arbitrary waits to stabilize tests.



12

Why Playwright?

Playwright's Superior Testability and Built-in Auto-Waiting Mechanism

- **Playwright's Auto-Waiting Solution**
- **Automatic checks for fundamental actions:**
  - Ensures the element is attached to the DOM.
  - Verifies the element is visible, enabled, and ready for interaction.
- **Enhances test stability and reliability by removing manual waits and checks.**
- **Intelligent Selectors for robust element targeting.**

13

Also:

Extensive Capabilities of Playwright:

- Device Emulation to simulate mobile and tablet environments.
- Geolocation & Time Zone Manipulation for location-specific testing.
- Permission Handling to manage browser permissions in tests.

Why Playwright ?

Ubiquitous

All major OSs:

- Linux, Mac, Windows

All major browsers:

- Chrome, Firefox, Safari (Webkit)

Many popular languages:

- JavaScript/TypeScript, Java, C#, Python

Full CI support

14

Supports Major Operating Systems: Playwright is compatible with various operating systems, including Windows, macOS, and Linux.Works Across Major Browsers: Playwright supports popular browsers like Chromium, Firefox, and WebKit, providing cross-browser testing.Flexible Language Support:

Tests can be written in several popular languages, such as JavaScript, TypeScript, Python, C#, and Java.CI Pipeline Integration: Playwright tests can run seamlessly in CI pipelines like GitHub Actions, Jenkins, and others.

Docker Container Support: It also runs efficiently within Docker containers, ensuring a consistent testing environment across different platforms.

Ubiquitous existing or being everywhere at the same time, existing or being everywhere at the same time : constantly encountered : widespread.*

Why Playwright ?

Delightful and lively

Debugging tools, smart code generation

Actively developed and maintained

15

Debugging and Code Generation

•Playwright offers **awesome debugging features** to help write tests faster.

•Includes **smart code generation**, which makes writing tests more efficient and quicker.

•When tests fail, Playwright aids in **root cause analysis**, enabling faster issue resolution.

Active Development and Maintenance

•Playwright is **actively developed and maintained**, ensuring continuous improvements.

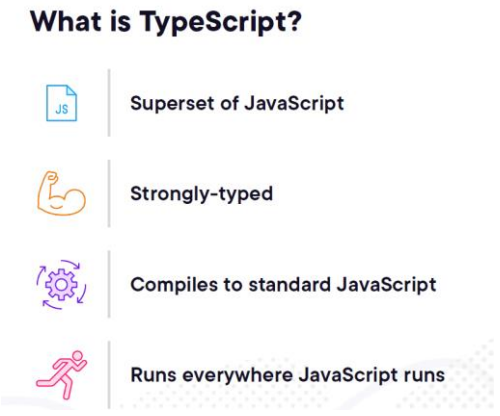
•Users can check the **community page** on the documentation website to stay updated on new developments.

- **Discord server**, **Dev.to blog**, and **YouTube channel** where new features are showcased with every release.

Introduction to TypeScript

TypeScript Fundamentals

- **TypeScript is a superset of JavaScript that adds static typing and other powerful features to enhance development efficiency and maintainability.**
- **It is widely used in modern web development, especially for large-scale applications.**



Superset of JavaScript:

•TypeScript builds on top of JavaScript, meaning all JavaScript code is valid in TypeScript.

•It extends JavaScript with additional features, making it more robust for development.

Strongly-Typed:

•Unlike JavaScript, TypeScript enforces **static typing**, which helps catch errors at compile time rather than runtime.

•This leads to **fewer bugs** and improved code reliability.

Compiles to Standard JavaScript:

•Browsers don't understand TypeScript directly, so it **compiles down to plain JavaScript** that can run anywhere.

•This ensures compatibility with **existing JavaScript frameworks and libraries**.

Runs Everywhere JavaScript Runs:

•Since it converts to JavaScript, TypeScript works in **browsers, Node.js, and any JavaScript-supported environment**.

•Developers can use TypeScript in both **frontend (React, Angular, Vue)** and **backend (Node.js, Deno)** development.

Pre-requisites of TypeScript

Setting up a Development Environment

Visual Studio Code
https://code.visualstudio.com/

Node.js
https://nodejs.org/

TypeScript Compiler
https://www.typescriptlang.org/

17

Basic Project Configuration

1. Choosing a Text Editor

- **Any text editor with TypeScript support will work.**
  ✔ Recommended: Visual Studio Code (VS Code).
  - Free and cross-platform (Windows, macOS, Linux).
  - Built-in TypeScript support.
    ✔ Download: code.visualstudio.com

2. Installing Node.js

- **Needed to run TypeScript programs.**
- **✔ Recommended Version: Latest Long-Term Support (LTS).**
- **✔ Download nodejs: https://nodejs.org/**
- **After installation:**
  - Run node -v to check your version.

18

Installing Node.js

3. Installing TypeScript

✔ **Official site: typescriptlang.org**
✔ **Multiple installation methods:**

- **npm (Recommended for cross-platform development)**
- NuGet (For Visual Studio users)
- Visual Studio Extension

4. Installing TypeScript via npm

✔ **Local installation (per project):**

- npm install typescript --save-dev
  - Allows different TypeScript versions for different projects.
  - Helps prevent breaking changes in production applications.

19

Installation process

Compiling and Running a TypeScript Program

✔Open VS Code in a new folder

✔Create a new terminal

- Ensure the project path is listed on the terminal
- Else , run the command: cd c:\<folder name>
- Type the command: npm install -g typescript
- Type the command: npm install -g ts-node

20

Installation Process

Running the Compiled JavaScript File

- **Initialize a project:**
  - Run the command: tsc --init
  - Creates a tsconfig.json file for compiler settings
- **Create a ts file and add the code:**

```
let newGreeting = "Hello TypeScript!";
console.log(newGreeting);
```

  - Run the program by writing the command: ts-node <classname.ts>
  - The terminal shall display Hello TypeScript!

21

Basic Syntax

Using let and Type annotations

- **TypeScript supports all JavaScript primitive types and adds a few more.**
  - Booleans: true / false.
  - Numbers: Floating-point values, same as JavaScript.
  - Strings: Can use single (‘) or double (").
  - Arrays: Primary data structure, similar to JavaScript.

22

Arrays

Understanding Arrays in TypeScript

- **Two ways to declare an array:**
  - Bracket Notation: Define the type followed by [].
  - Generic Syntax: Use Array<type> notation
- **Handling Multiple Types in an Array**
  - TypeScript allows arrays with multiple types using any, but this is not recommended.

```
let strArray1: string[] = ['here', 'are', 'strings'];

let strArray2: Array<string> = ['more', 'strings', 'here'];

let anyArray: any[] = [42, true, 'banana'];
```

23

To run the test; use 'ts-node <file name>'.

Note: **When to Use Which?**

- ✅ **Use string[]** for standard array declarations.
- ✅ **Use Array<string>** when dealing with complex generic types or functional programming

Loops in TypeScript

Types of loop

- **For Loop:**
  - Used when executing a block of code a fixed number of times.
    - Syntax:
      - Initialize a counter variable.
    - Condition:
      - Runs until the condition is false.
    - Increment/Decrement the counter.
- **While Loop:**
  - Runs as long as a condition is true.
  - The condition is evaluated before each iteration.

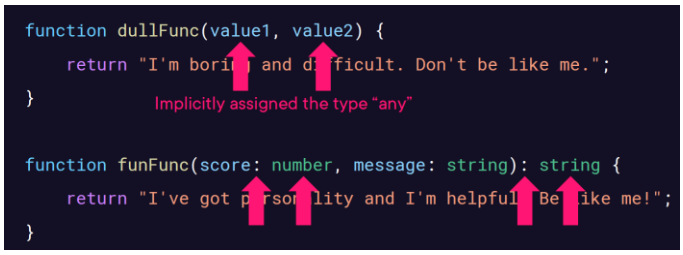
```
1 for (let i=1; i<=10; i++)
2 {
3   if (i % 2 == 0) {
4     console.log(`${i} - even`);
5   }
6   else {
7     console.log(`${i} - odd`);
8   }
9 }
```

24

Functions

Writing a Function

- Functions in TypeScript use type annotations for parameters and return values.
- Example:



- Implement the below function:

```
function PrintMovieInfo(title: string, yearReleased: number) {
   console.log(title, yearReleased);
   }
```

- To call a function, write : PrintMovieInfo('A New Hope',2000)

25

Exercise

Calculating Payroll Deductions

1. **Write a function that calculates net salary after deductions:**
   - Income Tax: 15%
   - Pension Contribution: 5%
   - Deducted amount should be rounded to 2 decimal places.

26

Calculating Payroll Deductions

Solution

```
function calculateNetSalary(salary: number): number {
  const tax = salary * 0.15;
  const pension = salary * 0.05;
  return parseFloat((salary - tax - pension).toFixed(2));
  }

  console.log(calculateNetSalary(5000)); // Expected Output: 4000
```

27

Block-scoped Variable

Let

- let allows reassignment but is block-scoped (available only inside the block {} where it's declared).
- Prevents accidental redeclaration within the same scope.
- Example:

```
let name = "Alice";
console.log(name); // Alice
name = "Bob";  // ✅ Allowed (Reassignment is possible)
console.log(name); // Bob
```

28

Block Scope Example:

```
if (true) {
   let age = 25;
   console.log(age); // 25
}
console.log(age); // ❌ Error: age is not defined (because `let` is block-scoped)
```

Block-scoped Variable

const

- const CANNOT be reassigned after declaration.
- However, objects declared with const can have their properties modified.

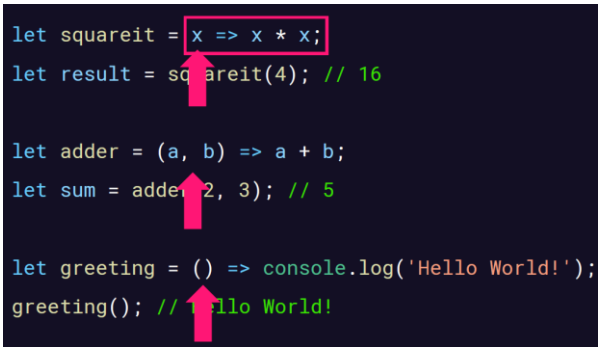
```
const country = "USA";
console.log(country); // USA

country = "Canada"; // ❌ Error: Assignment to constant variable
```

29

Arrow Functions

Anatomy of an arrow function

- Arrow functions were introduced in ES2015 and adopted by TypeScript.
- Also known as lambdas in other languages.
- They are anonymous functions with a simplified syntax.
- Syntax Overview
  - General form:
  - Parameters on the left.
  - Arrow (=>) in the middle.
  - Function body on the right.
  - Sometimes called "fat arrow" functions because of the => symbol.



30

Note: **ES2015 (ECMAScript 2015)**, also known as **ES6**, is a major update to the JavaScript language that introduced many new features and improvements. It was released in **2015** and is one of the biggest updates to JavaScript.

Example:

Template Literals (``)

Allows embedding variables inside strings using ${}.

```
const name = "Alice";
console.log(`Hello, ${name}!`); // Hello, Alice!
```

Arrow Functions

Declaring an arrow function:

- **let squareIt = x => x * x;**
  - x is the parameter.
  - => separates the parameter from the function body
  - .x * x is the function body.
- **Multi-line Arrow Functions**
  - If the function body has more than one line, use curly braces {} and an explicit return

```
let add = (a, b) => {
   let sum = a + b;
    return sum;
};
```

31

Arrow Functions Practice

Simple Function vs Arrow function

- A basic function that takes parameters and returns a value can be converted easily to an arrow function.
- Normal Function:

```
1. function add(a: number, b: number): number {
  return a + b;
}
2. function calculateArea(length: number, width: number): number {
  const area = length * width;
  return area;
}
```

- Exercise: Convert these normal functions into arrow functions.

32

Answer: const add = (a: number, b: number): number => a + b;

Answer: const calculateArea = (length: number, width: number): number => {
  const area = length * width;
  return area;
};

Classes in TypeScript

Introduction to Classes

- They serve as templates for creating objects with predefined properties and methods.
- Essential for object-oriented programming (OOP) in TypeScript.
- Key Characteristics of Classes
  - Define types that structure objects with properties and methods.
  - Provide state storage through properties to hold object data.
  - Offer behavior through methods to define object actions.
  - Enable encapsulation by grouping related functionality.
  - Support reuse and maintainability by organizing code logically.

33

Class Members in TypeScript

•**Constructors**: Special methods to initialize objects.

•**Properties**: Variables that store object data.

•**Methods**: Functions that define object behavior.

Inheritance in TypeScript

•Allows a class to **extend** another class, inheriting its properties and methods.

•Enables **code reusability** by creating a hierarchy of related classes.

•Uses the extends keyword to create **subclasses**.

Access Modifiers

•**Public**: Accessible from anywhere.

•**Private**: Accessible only within the class.

•**Protected**: Accessible within the class and its subclasses.

Defining a class

- A class is a blueprint for creating objects.
- Declared using the class keyword.
- Contains properties (variables) and methods (functions).
- Example:

```
class Person {
 name: string;
 age: number;
 constructor(name: string, age: number) {
  this.name = name;
  this.age = age;
 }
 greet() {
  console.log(`Hello, my name is ${this.name}`);
 }
}
```

34

Constructors in TypeScript

What is a Constructor?

- A constructor is a special function that initializes new instances of a class.
- Defined using the constructor keyword inside a class.
- Can take parameters to initialize class properties.
- A class can have only one constructor.

Example:

```
constructor(name: string, age: number) {
   this.name = name;
   this.age = age;
 }
```

35

Note: Allows initialization of properties when an object is created.

Unlike some OOP languages, TypeScript does not support multiple constructors.

Can achieve similar behavior using optional parameters.

Example:

constructor(title: string, publisher?: string) {}

The ? makes publisher optional.

Custom Accessors

Getters & Setters

- Provide controlled access to properties.
- Defined using get and set keywords.
- Example:

```
class Book {
 private _editor: string = "";

 get editor(): string {
  return this._editor;
 }
 set editor(newEditor: string) {
  this._editor = newEditor;
 }
}
```

36

Modules

Why Use Modules in TypeScript?

- **Encapsulate implementation details within a module.**
- **Promote reusability—modules can be used across different parts of an app or even in other applications.**
- **Using Modules in TypeScript – Syntax Overview**
  - Exporting a Module

```
export function greet(name: string) {
  return `Hello, ${name}!`;
}
```

  - Importing a Module

```
import { greet } from './greetings';
console.log(greet("John"));
```

37

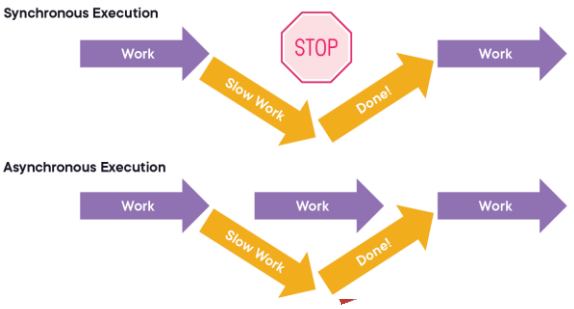
- Use **named exports** or **default exports** based on use case.
- Import syntax depends on module system and target environment.

Writing Asynchronous code

Introduction

- **Synchronous Execution Problem**
  - Executes tasks one at a time.
  - Issue: Long tasks (e.g., network calls) block execution.
  - Result:
    - App appears frozen.
    - Poor user experience.

- Asynchronous Execution Solution
  - Long tasks run in the background.
  - Main thread remains free to handle other tasks.
  - **Result**: Smooth user experience and increased efficiency.



38

Playwright Installation Guide

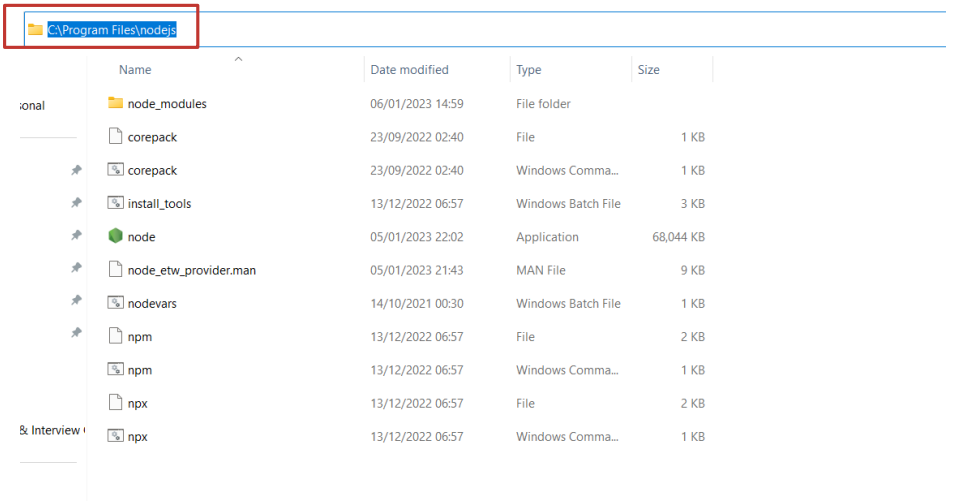
Pre-Requesites

1) **Browse to Node.js website and click on downloads - https://nodejs.org/en/download/.**

2) **Download Visual Studio Code - https://code.visualstudio.com/ .**

Steps to set Node.js globally on laptop

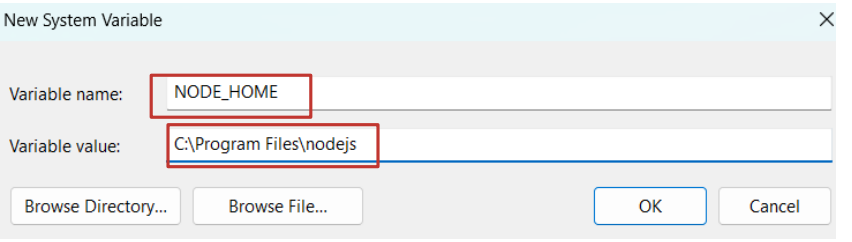
1) Navigate to Program Files and open nodejs folders and copy the path.



41

Steps to set Node.js globally on laptop

2) Open Environment Variable and add new system variable.



42

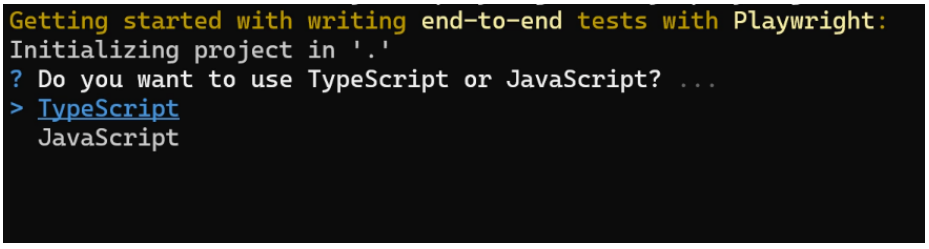
Create npm Project and install Playwright dependencies for testing

1. Download the sample project playwright-nodejs-starter-main.
2. Extract the project files and save them in the C: drive.
3. Open Visual Studio Code and navigate to the playwright-nodejs-starter-main folder.
4. Open a terminal in VS Code, then install Playwright by running the following command and pressing Enter:npm init playwright@latest

43

Create npm Project and install Playwright dependencies for testing

4. Select (y) to proceed and select TypeScript.



44

Select **TypeScript** instead of **JavaScript** when prompted. This will generate project files with a **.ts** extension, but you can still write **pure JavaScript** if preferred.

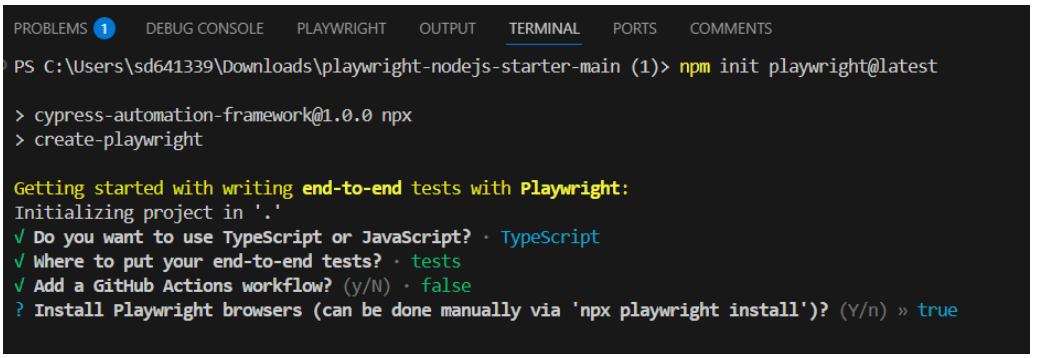
Next, accept the default configurations, including the **test directory**, but **do not add GitHub Actions**, as **CI integration is beyond the scope of this course**.

Press **"Yes"** to install **Playwright browsers**, as they are required before running your first test.

This setup will generate a **package.json** file—something you may already be familiar with if you've previously worked with **Node.js projects**. It will automatically pull in the required **dependencies** and install the latest available **browsers**.

Create npm Project and install Playwright dependencies for testing

5. **Select those settings as below.**

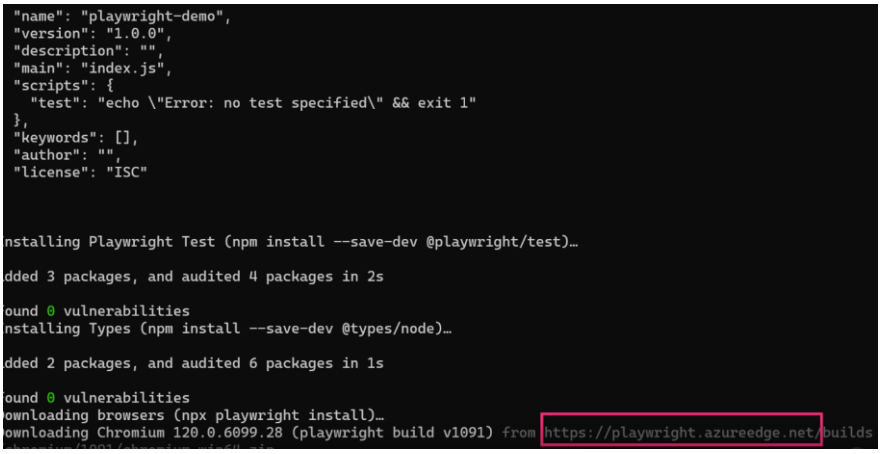


45

By following the steps above, you will successfully set up a **Playwright project**, and all necessary **dependencies** will be automatically installed, ensuring a smooth development experience.

Create npm Project and install Playwright dependencies for testing

5. Playwright downloads the web browsers



46

By default, **Playwright downloads browsers from Microsoft's CDN**, which is a specialized and secure server. However, if you are behind a corporate firewall, this operation **may be blocked**. In such cases, refer to the **Playwright documentation** under the section **"Install behind a firewall or a proxy"** for guidance on bypassing restrictions.

Playwright does not download Google Chrome. Instead, it downloads **Chromium**, an open-source web browser developed by Google. Chromium serves as the foundation for browsers like **Edge, Opera, and Chrome**.

The playwright installed files

This is a sample set of tests provided by playwright.

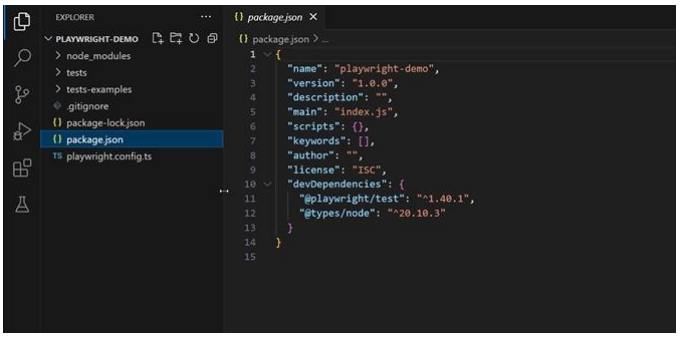
- **Key Roles of package.json in Playwright**

  ✔ Manages dependencies (e.g., Playwright, TypeScript, Cucumber).

  ✔ Defines scripts for running tests and other automation tasks.

  ✔ Stores metadata like project name, version, and author.

  ✔ Specifies configurations like Jest/Playwright settings.



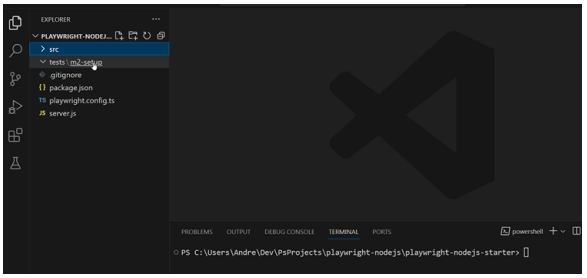
47

What is package.json in Playwright?

The package.json file in a **Playwright** project is a configuration file used to manage dependencies, scripts, and metadata related to the project. It is essential for running and managing Playwright tests.

What to Expect After Opening the Project?

- **Once opened in VS Code, the project should contain:**
  - The source directory with a practice website that runs on a local server.
  - The test directory, organized with subdirectories for each module.
  - The package.json file, listing all dependencies.
    - Dependencies include Playwright, TypeScript, and Express (a popular web framework).
    - The start script runs server.js, which uses Express to start a basic local server.



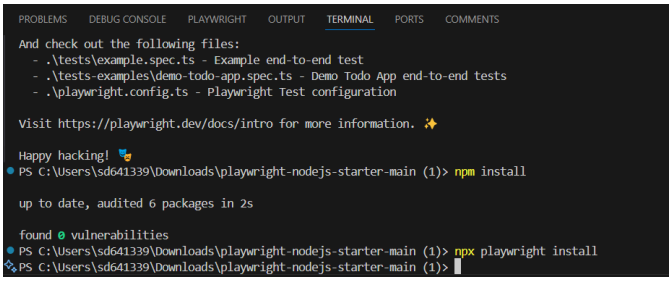
To initialize the project, run a few **npm commands** from the **integrated terminal** in **VS Code**.

⚠ **Note:** Depending on your system setup, you **might encounter permission issues**. If any commands **fail**, try running them in your **OS's native terminal (Command Prompt, PowerShell, or Bash).**

Installing dependencies

Ensure you are in your project directory, then run the following: command:

- **npm install**
  - This will install all the required dependencies, including Playwright.
  - The node_modules folder should appear at the top of your project structure.



- **Next, install the Playwright browsers by running: npx playwright install**
  - This will download the necessary browsers unless they are already installed, in which case it will skip the installation.

49

npm (Node Package Manager):

•**Purpose**: npm is primarily used to manage packages in Node.js projects. You use npm to install, update, and manage dependencies in your project.

•**Common Uses**:

- npm install <package>: Install a package and add it to the node_modules folder.
- npm install: Install all dependencies listed in the package.json.
- npm run <script>: Run a script defined in the package.json.
- npm init: Initialize a new Node.js project by creating a package.json file.

npx (Node Package Executor):

•**Purpose**: npx is a command-line tool that comes with npm (since version 5.2.0). It allows you to run binaries from Node modules or even packages that are not installed globally or locally in your project.

•**Common Uses**:

- npx <command>: Run a command without needing to install the package globally or locally. It temporarily installs and executes the package for the duration of the command.
- npx create-react-app my-app: This is an example of running a tool (create-react-app) without needing to install it globally.
- npx --version: Check the version of npx.

Key Differences:

1.Global Installation:

1. npm often requires installing packages (locally or globally) before you can run them.
2. npx allows you to run commands without having to install them first.

2.One-off Command Usage:

1. npx is handy for running a package or command just once without the need to install it, making it more lightweight for temporary tasks.

3.Package Execution:

1. npm installs packages, while npx can run commands from those packages directly without installation.

Example:

•If you need to use create-react-app:

- With npm, you'd first install it: npm install -g create-react-app, and then use it.
- With npx, you can run it directly: npx create-react-app my-app.

Create a first JS file to test our installation and configuration

- **Write a JavaScript function.**
- **Import chromium as the web browser.**
- **Write the IIFE function and run the test.**
- **This will ensure that the configuration was done correctly.**

At the beginning of the script, the chromium object is required from the @playwright/test dependency. For initial setup, only the Chromium browser is used, which is sufficient to verify that everything is configured correctly.

The script is structured as an **Immediately Invoked Function Expression (IIFE)**—a self-executing function that runs as soon as it is defined. This pattern, though bracket-heavy, helps keep the code clean and isolated.

Key Steps in the Script:

•**Launching Chromium:** Playwright is used to launch a new instance of the Chromium browser, allowing interaction with the browser.

•**Closing the Browser:** Once the necessary actions are completed, the browser instance is closed to free up system resources.

•**Logging Confirmation:** A message is displayed in the console, confirming that the script executed successfully.

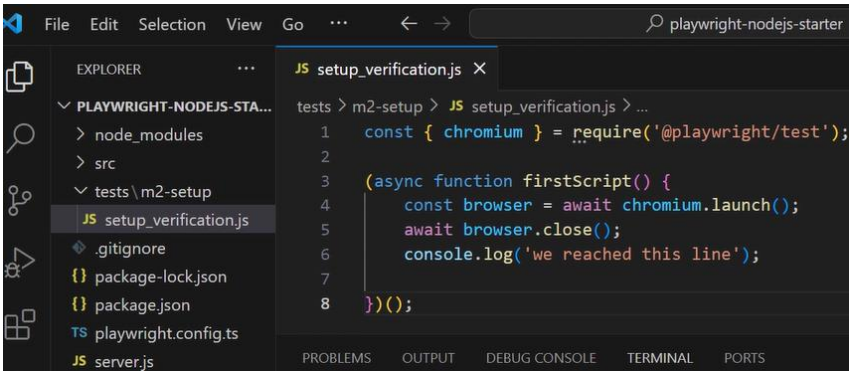
Reaching the end of the script without errors indicates that:
✔ The Playwright setup is correctly configured.
✔ Playwright's API can be used for browser automation (in this case, Chromium).

This serves as the foundation for further automation using Playwright.

Verification of configuration done so far

Execute a JavaScript file

- **In VS Code, open the terminal and run:**
  - node .\tests\m2-setup\setup_verification.js



51

Once executed, you should see the **console messages** printed without any errors. This confirms that the **browser launched and closed successfully**.

You might notice that no actual browser window appears—this is because Playwright runs in **headless mode by default** (without a visible UI).

Introduction Demo Project

Sample Project Overview

- The src folder contains HTML, CSS, and JavaScript files.
- The index.html file represents a credit union website with three main pages:
  - Register
  - Deposit & Earn Interest
  - Apply for a Loan
- Local vs HTTP Server
  - Initially, the project runs as a local file, visible in the browser's address bar.
  - Ideal setup: Run it on a local HTTP server for better testing capabilities.
- Setting Up a Local Server
  - A simple Express.js server (server.js) is used.
  - Defined in package.json as the start script (npm start).
  - Running npm start launches the local server on localhost.

52

Playwright Automation

•The **global config file** automates server startup:

- Defines a webServer command.
- Runs npm start before tests.
- Shuts down automatically after tests.

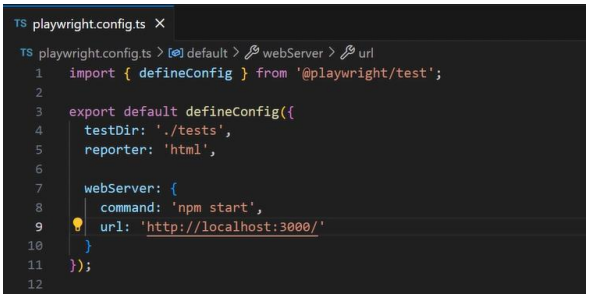
•**npx playwright test** execution:

- Reads the **webServer command**.
- Runs npm start, which executes server.js to serve the project.

Setting up Global configuration for playwright

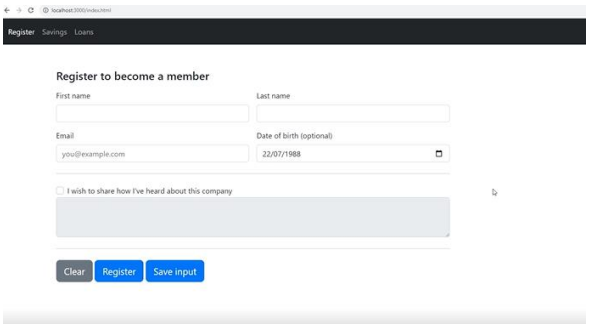
Playwright.config.ts file:

- **The global config file automates server startup:**
  - Defines a webServer command.
  - Runs npm start before tests.
  - Shuts down automatically after tests.
- **npx playwright test execution:**
  - Reads the **webServer command**.
  - Runs npm start, which executes server.js to serve the project.



Sample project - Running Tests Locally with a Web Server

- **server.js runs the tests locally on your machine.**
- **The package.json file references server.js in the start script.**
- **Running npm start launches a local server.**



🔗 **Accessing the Local Server:**

•Open http://localhost:3000/ to view the site served locally over HTTP.

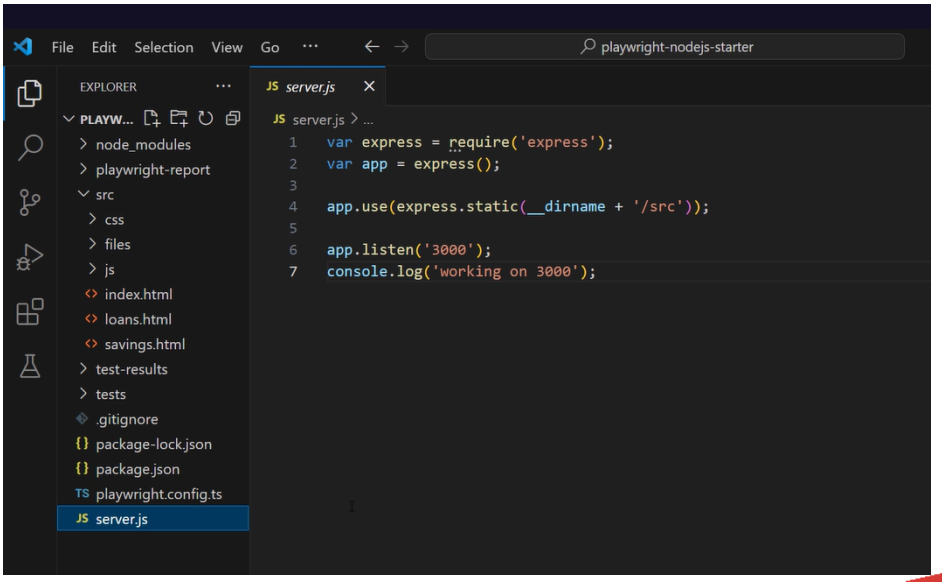
🚀 **Automating This in Playwright:**

•Instead of manually starting the server, define a **webServer** in playwright.config.

•Set the URL to **http://localhost:3000/**, allowing Playwright to navigate directly.

The sample project files

Server.js file



55

Because we are accessing it as a file on local storage, this is what we see in the address bar.

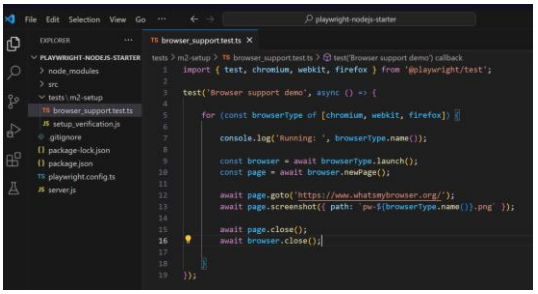
We can navigate to local files with Playwright, but ideally, we should have it running on a local HTTP server.

This is why a very simple local server with the Express framework is created.

We point to the directory with the website and specify the port. Note that this code is placed in the server.js file, and this server.js file is specified in package.json as the start script which means that we can open up a separate command shell at the root of the project and run **npm start**. It should start up a local server.

Writing a second test script file in TypeScript

- **To run the test, use npx playwright test tests/m2-setup/browser_support.test.ts**
- **Launch the browser, create a page, and navigate to a site that detects the browser.**
- **Take a screenshot, save it, and close resources.**



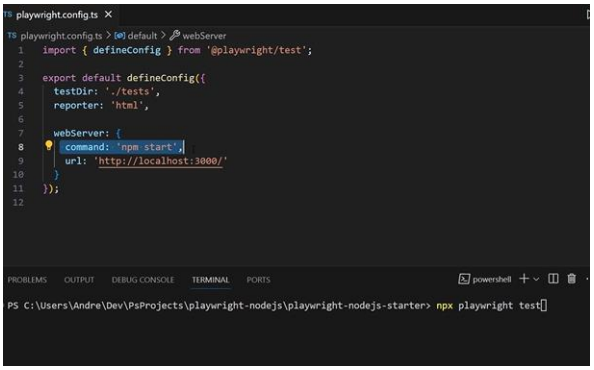
56

Playwright supports three main browser engines: **Chromium, WebKit, and Firefox**.Instead of plain JavaScript, create a **TypeScript test file** (e.g., example.test.ts).Use **ES module syntax** (import instead of require).Import test and browser types (Chromium, WebKit, Firefox).**Structure of the test**:Iterate over each browser type.

When selecting a file, Windows auto-completes paths with backslashes (\), which work for single files. However, using backslashes for directories results in a syntax error (Invalid Regular Expression). ✅ Solution: Use forward slashes (/) to ensure Playwright correctly finds and runs the tests.

Playwright global configuration file

- **playwright.config.ts file**
  - The global configuration file allows specifying the webServer setting.
  - The webServer setting enables running a specific command and navigating to a designated URL.
  - When executing the npx playwright test command, Playwright automatically starts the local server.
  - The server shuts down once the test execution is complete.
  - Before running tests, Playwright detects the webServer command in the configuration file.
  - This setup runs npm start, which in turn executes node server.js.node server.js serves as a minimal server to host the project.



2. Getting Started

RUBRiC

- In this lesson you will learn about:
  - 1.2 Playwright core concepts
  - 1.2.1 Element locating strategies
  - 1.2.2 Basic and advanced actions
  - 1.2.3 Playwright Test Runner
  - 1.2.4 Functionality through configuration
  - 1.2.5 Tooling for troubleshooting and test creation

58

Structuring and Running Playwright Tests

📌 **Key Takeaways from the Last Module:**

1 From IIFE to Proper Test Functions

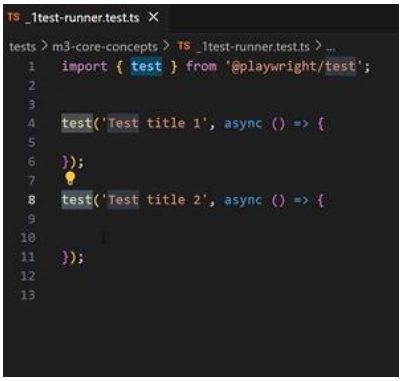
- Initially, we wrote Playwright code inside an IIFE (self-executing function).
- We then transitioned to proper test functions, the recommended structure for Playwright tests.

2 Multiple Tests in a Single File

- A test file can contain multiple tests, but each test must have a unique name.
- For Playwright's CLI to recognize a test file, its name must contain test or spec.

3 Running Tests with Playwright CLI

- Running **npx playwright test** executes all tests.
- To run a specific test file, use tab auto-completion.



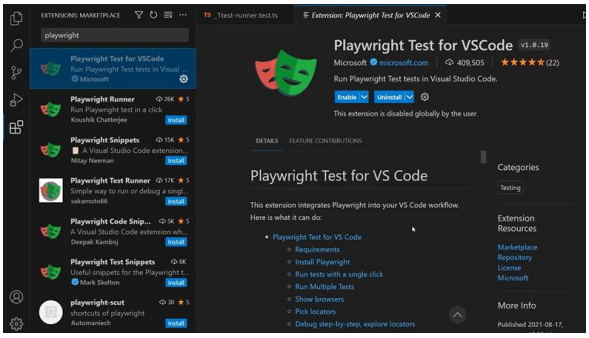
59

Note: File Path Considerations on Windows:

When selecting a file, Windows auto-completes paths with backslashes (\), which work for single files. However, using backslashes for directories results in a syntax error (Invalid Regular Expression). ✅ Solution: Use forward slashes (/) to ensure Playwright correctly finds and runs the tests. 🚀

Playwright Test Runner Extension:Playwright Test for VS Code

- **Seamless Test Execution: This extension allows you to efficiently run tests directly from your IDE.**
- **Static Code Evaluation: The plugin does not automatically reevaluate source code changes.**
- **Manual Refresh Required:**
  - If you add a new test to an existing file, it won't appear immediately.
  - To detect the new test, you must:
    - Close and reopen the fileRefresh the test explorerRestart the IDE

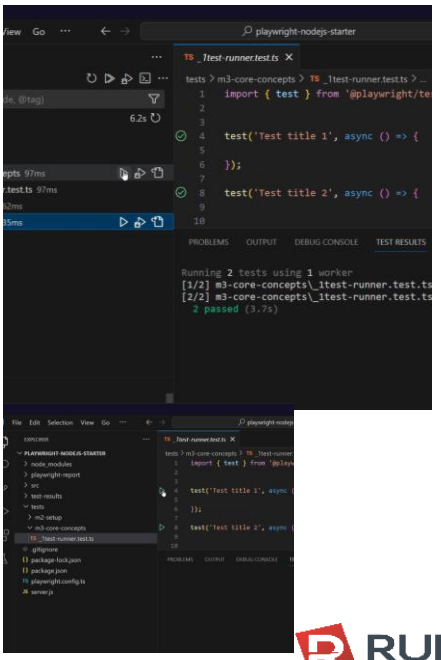


Best Practice: Always refresh your test environment after modifying test files to ensure Playwright recognizes all changes.

Installing VS Code Plugin :Playwright Test for VS Code

Once the test runner extension is installed, green arrows appear next to every test within each file. This allows for seamless test execution:

- A single test can be executed with a left-click.
- Right-clicking provides advanced run options, such as running tests in debug mode.
- To execute multiple tests, the Test Explorer can be used. This icon appears automatically when a test runner extension is installed.



61

Test Execution Options in Test Explorer

From the Test Explorer, it is possible to:

- ✅ Run a single test
- ✅ Execute all tests in a specific file
- ✅ Run all tests within a particular directory
- ✅ Execute all tests in a top-level directory

Limitations and Workarounds

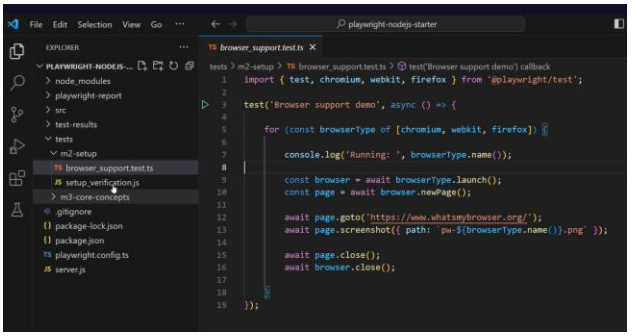
The plugin does not re-evaluate source code on every change. If a new test is added to an existing file, the plugin does not automatically detect it. To resolve this:

- 🔄 Close and reopen the file
- 🔄 Click on the testing icon and select **Refresh Tests**

These features streamline the testing workflow, offering flexibility and control over test execution.

Fixtures

- **Manually calling chromium.launch and browser.newPage in every test is tedious but necessary for opening a browser and page.**
- **Playwright simplifies this process with built-in fixtures, handling browser and page creation automatically.**



Using the Page Fixture

✅ Adding mandatory curly brackets and the page fixture allows Playwright to manage the browser and page.
✅ Eliminates the need for manual setup and teardown code.
✅ Results in cleaner, more maintainable test scripts.
✅ Running the test fetches and prints Playwright's homepage title.

Handling Multi-Page Tests with the Context Fixture

•For tests requiring multiple pages, the **context fixture** is used:

```
test('other fixtures', async ({ context }) => {
                const page1 = await context.newPage();
                const page2 = await context.newPage();
});
```

Simplifying Test Setup with Playwright Fixtures

- **Using the page Fixture**
  - Instead of manually setting up and tearing down the browser, we can use Playwright's page fixture, which automatically:
    - ✅ Launches a browser
    - ✅ Opens a new page
    - ✅ Handles cleanup
  - Refactoring the CodeBy importing { page }, we eliminate the need for explicit browser setup and teardown. This makes the test shorter, cleaner, and easier to maintain.
    - Less writing
    - Less reading
    - Less code maintenance

Does Playwright Run a Default Browser?

- **Yes! Playwright runs a default browser if none is specified.**
- **The default browser is Chromium.**
- **When using page, Playwright automatically launches Chromium.**

- **How to Specify a Different Browser?**
  - Run tests in a specific browser via CLI:
  - npx playwright test --browser=firefox
  - Or Set a default browser in playwright.config.ts:
  - import { defineConfig } from '@playwright/test';

```
export default defineConfig({
  use: {
    browserName: 'firefox' // Options: 'chromium', 'firefox', 'webkit'
  }
});
```

Understanding Async and await

- **In programming, it is often necessary to wait for a task to finish before proceeding.**
- **The async and await keywords help make the code both functional and simple.**
- **Async signals that a part of the code may take time, allowing other tasks to run in the meantime.**
- **Await is used when the result of an action is required before continuing execution.**
  - Example: Navigating to a page takes time, so the test must wait for it to finish loading.
  - Await should be used when a function returns a promise to ensure proper execution flow.

```
import { test, expect } from '@playwright/test';

test('Simple assertions', async () => {

    expect('a').toEqual('a');

    expect(2).toBeLessThan(3);
    expect(null).toBeTruthy();


});


test('Test with simple auto-retrying assertions', async ({ page }) => {

    await page.goto('http://localhost:3000/');

    await expect(page).toHaveTitle('Credit Association');
    await expect(page).toHaveURL('http://localhost:3000/');

});
```

Note: Most functions return a promise and some do not. This is why we need to use async with await.

Async and await

- It is similar to pausing and waiting until a required process is complete.
- Navigating to a page takes time, so waiting for it to finish loading is necessary.
- Locating an element before performing an action requires time.
- Verifying that something appears on the page may also introduce delays.
- Any operation that takes time must be awaited.

Things That Take Time

```
await page.goto('url');
await page.getByRole('button').click();
await expect(result).toContainText('...');
```

```
// This takes time...
async function fetchData() {...}

// wait before moving on
const data = await fetchThing();
```

Note:

•The top-level function must be marked as async if it contains an await statement.

•Removing async from another test causes the IDE to immediately highlight syntax issues.

Using Expect

Assertion in tests

- **Fixtures, especially the page fixture, help maintain clean and efficient code.**
- **So far, the focus has been on printing output, but a complete test requires verification or assertions using Playwright's expect library.**
- **To implement assertions, the expect function must be imported.**
- **A major cause of flaky UI tests is temporary page delays.**
  - If a test executes too quickly, it may fail prematurely, even though the correct behaviour appears shortly after.
  - This results in a race condition, where an assertion happens milliseconds too soon, while the expected state manifests slightly later.

Writing Playwright tests

Guidelines

- **Organize Tests Properly**
  - Place each test inside a test function.
  - Ensure test titles are unique within the same test file.
- **Reduce Boilerplate Code**
  - Use page and other fixtures to simplify setup and teardown.
  - Ensure Stability in Tests
    - Always use await for actions and assertions that return a promise.
    - Helps prevent flaky tests and ensures proper execution order.

```
test('Unique test name in file', async ({ page }) => {

   await page.goto('url');

   // with retries
   await expect(page).toHaveTitle('Welcome');

});
```

68

Run all tests using command line: **npx playwright test,**

Run a specific test file using this command: **npx playwright test tests/your-file.spec.ts**

Run a set of tests using the command: **npx playwright test tests/your-dir/**

Locators

Commonly Used Playwright Locators

- Saving a locator is useful when performing multiple actions on an element.
- When using getByRole, specifying the tag and attribute name is recommended.
- The getByText function is ideal for locating elements based on inner text within a .
- These are the recommended locators:
  - Using getByLabel()
  - Using getByRole()
  - Using getByText()

```
test('Recommended built-in locators examples', async ({ page }) => {
    await page.goto(''); – 1350ms

    const firstName = page.getByLabel('First name');
    await firstName.fill('Sofia'); – 1134ms
    await firstName.clear(); – 1073ms

    await page.getByLabel('First name').fill('Andrejs'); – 1041ms

    await page.getByRole('button', { name: 'Register', exact: true }).click(); – 1149ms

    const warning = page.getByText('Valid last name is required');

    await expect(warning).toBeVisible(); – 19ms
```

69

Full documentation: [Locators | Playwright]

Locating elements in a web page

HTML web page

- A web page contains various elements such as inputs, checkboxes, links, and buttons.
- Locating these elements is essential before performing any actions, as they are defined by HTML tags.
- Playwright offers multiple selection methods, with the best choice depending on the context.

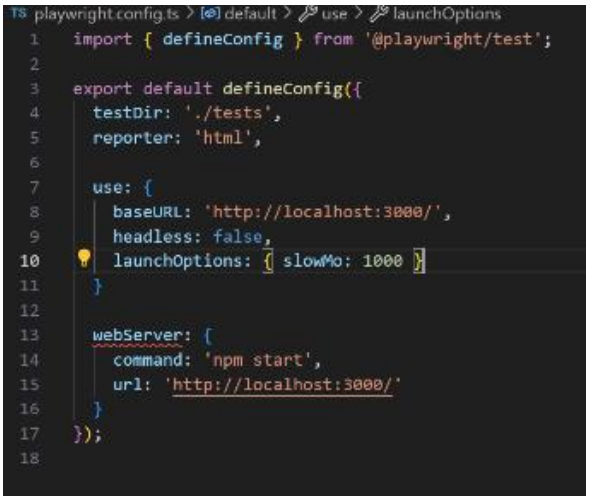
```
<form>
    <div class="form-group">
        <label for="nameInput">Name</label>
        <input type="text">
    </div>
    <button type="submit">Submit</button>
</form>
```

70

Slowing things down

Playwright config ts

- **Set a base url.**
- **Headless: false**
- **launchOptions: {slowMo:1000}**



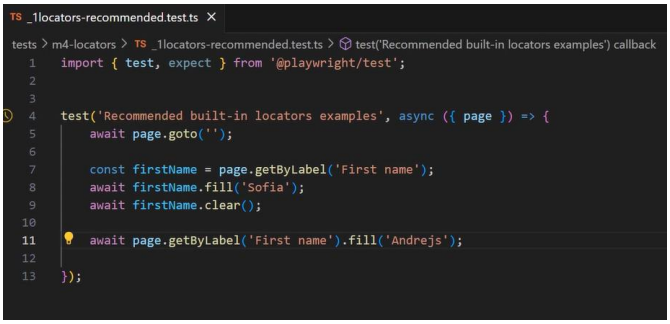
71

It's beneficial to observe processes at a reasonable speed while learning or debugging.
To facilitate this, some default settings in the global configuration file will be overridden.

Writing tests with locators

Forms

- A form with inputs is arguably one of the most common elements found on a website.
- Nearly all user actions return a promise, so they should be awaited.
- Saving the locator is beneficial when performing multiple actions on the same element.
- If the element is needed only once, the user action can be chained in a single line.
- To determine what other options can be specified:
  - Refer to the documentation.
  - Use Ctrl+Click on the function to explore supported elements.



72

A form with inputs, arguably one of the most common things you can find on a website.

Pretty much all user actions return a promise, so we should await them. Saving the locator is useful if you plan to perform two or more actions on the element. But if you only need it once, then you can just chain the user action like so in one line.

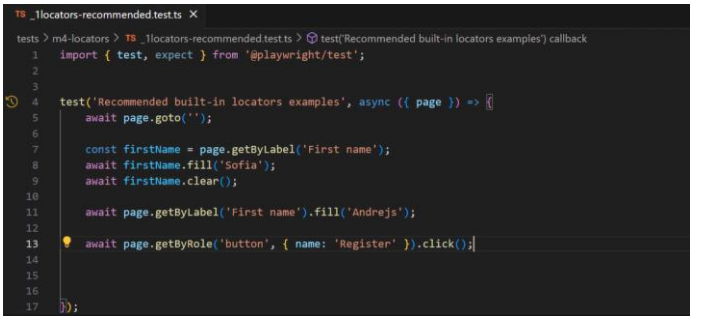
So how do we know what other things we can specify here?

We can read the documentation or Ctrl+click into the function and see all the different elements that are supported.

Writing tests with locators

Buttons

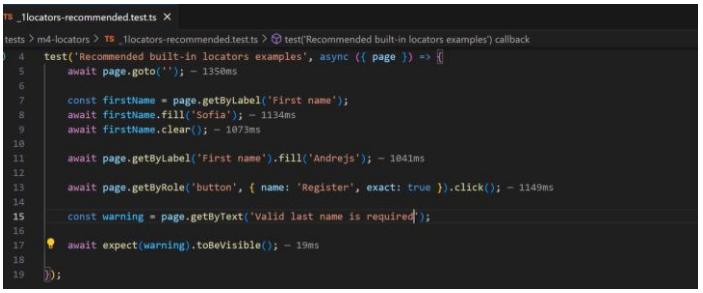
- Forms are not the majority of a website—various common elements exist, such as:
  - Links, buttons, checkboxes, lists, tables, and more.
- To interact with a button, it must first be located.
- For example, consider clicking the Register button that triggers form validation.
- Simply specifying "button" as the first argument is too generic since multiple buttons may exist on the page.
- To target the correct button, use a more specific selector, such as the name attribute with the value Register.



Writing tests with locators

Choosing the Right Locator for Element Selection

- Since the element is placed inside a generic , the getByText function is the ideal choice.



Generic Locators

Best Practices for Locating Elements

- It is highly recommended to:
  - Use specialized locators (getByRole, getByText, getByLabel, etc.).
  - Collaborate with developers to insert test IDs for automation.
- In some cases, none of these approaches may work.
- As a last resort, CSS and XPath selectors can be used.
- Extracting XPath and CSS selectors is a separate topic but should only be relied upon when necessary.

Applying Filters

Handling Identical Elements in Complex Web Applications

- Real-life web applications are often more complex than dummy projects.
- Many elements may have identical boxes, buttons, and text.
- Recommended locators like getByRole may match multiple elements.
- Sometimes, only a specific instance—such as the second or third element—is needed.
- Instead of using complex XPath expressions, Playwright provides the filter function for precise selection.

```
<ul>
  <li>
    <p>Item 1</p>
    <button>Add</button>
  </li>
  <li>
    <p>Item 2</p>
    <button>Add</button>
  </li>
</ul>
```

76

Applying Filters

Filtering Rows and Selecting Specific Cells

- First, retrieve all rows and apply a filter to select the one containing the text "competition".
- Save the locator and print its textContent.
- Running the test confirms that only one element—the last row—is correctly filtered and fetched.
- Selecting a Specific Cell
  - To get a single cell within the row:
    - Retrieve all rows.
    - Filter to select a specific row.
    - Within that row, locate a cell using getByRole.
    - Since multiple cells may match, use nth-child(1) to select the second cell (indexing starts at 0).
  - Running the test should print "2%", confirming the correct selection.

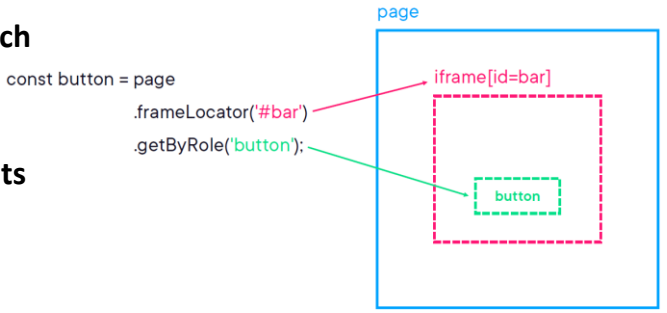
```
2
3   test('Filtering demo', async ({ page }) => {
4
5       await page.goto('/savings.html');
6
7       const rows = page.getByRole('row');
8       console.log(await rows.count());
9
10      const row = page.getByRole('row')
11          .filter({ hasText: 'Competition' });
12
13      console.log(await row.textContent());
14
15
16      page.getByRole('row')
17          .filter({ hasText: 'Competition' })
18          .getByRole('cell')
19          .nth(1);
20
21  });
```

77

Locating IFrames

Handling iFrames with Ease in Playwright

- **While iFrames are still widely used, they don't require a complex approach in Playwright.**
- **The frameLocator function makes locating and interacting with elements inside an iFrame straightforward.**
- **Once inside the frame, everything works the same as with top-level elements.**



78

For more details, refer to the Playwright documentation on Frames.

Codegen

- **In VS Code, click on the extension: Playwright Test for VS Code.**
- **Click on 'Record New'.**
- **A new browser window should spin up.**
- **Record the necessary actions.**
  - Navigate to a page, click and type as a normal user would.
- **Once finished, close the window and stop the process.**
- **A new file is generated.**
- **Playwright makes an effort to use robust locators. It uses a variety of functions such as; getByLabel, getByText, and getByRole**

79

•Manually writing locators can be tedious, but the **VS Code Playwright plugin** helps speed up the process.

•If installed earlier through the **Extensions Marketplace**, an icon should appear in VS Code.

•At the bottom, look for **"Record new"** and click it.

•This will launch a new browser window, allowing you to **record actions** instead of manually writing locators.

Recap: Best Practices for Locating Elements

•Prefer **user-facing locators**, such as:

- getByLabel
- getByText
- getByRole

•If these do not work, fall back to the **generic locator function**.

Playwright Basic Actions

Handling Identical Elements in Complex Web Applications

You may already be familiar with performing basic actions:

- Navigate using page.goto().
- Create a locator.
- Execute an action like locator.click(), locator.fill(), locator.check(), or locator.select().

However, there are additional configuration options and pitfalls to consider.

```
import { test, expect } from '@playwright/test';

const homeTitle = 'Credit Association';
const savingsTitle = 'Save with us';

test('Back, forward, reload (refresh) test', async ({ page }) => {

    await page.goto('/'); – 73ms

    await page.goto('/savings.html'); – 40ms
    await expect(page).toHaveTitle(savingsTitle); – 31ms

    await page.goBack(); – 57ms
    await expect(page).toHaveTitle(homeTitle); – 24ms

    await page.goForward(); – 35ms
    await expect(page).toHaveTitle(savingsTitle); – 31ms


    await page.reload(); – 39ms
    await expect(page).toHaveTitle(savingsTitle); – 30ms
});
```

Exploring Navigation Actions

- **Users frequently perform navigation actions such as:**
  - page.goBack()
  - page.goForward()
  - page.reload()
- **To add meaningful assertions, use expect statements to verify page titles after each action.**
- **Running the test should confirm that everything works as expected.**
- **Advanced Configuration**
  - Every navigation action accepts a second optional parameter to configure specific expectations.
  - This allows greater control over behavior, ensuring stability in test execution.

Handling Date Inputs and Formatting in Playwright

- **When filling in a date input, ensure the format aligns with the expected standard.**
- **For example, entering October 10th in different formats may seem correct, but it can still fail.**
- **Running the test might result in an error: "Malformed value."**
- **Why Does This Happen?**
  - The underlying system requires the ISO 8601 format (YYYY-MM-DD).
  - This is the internationally recognized standard for date parsing.
- **Solution:**
  - Always input dates in the YYYY-MM-DD format to avoid parsing errors.

Simulating Multiple Clicks and Advanced Click Options

- **The click() function is straightforward but can be extended for multiple clicks on the same element.**
- **Ways to Perform Multiple Clicks:**
  - Copy-paste the click() function multiple times (inefficient).
  - Use a for loop to repeat the action dynamically.
  - Use the clickCount parameter (cleanest approach).
- **Example:**
  - await locator.click({ clickCount: 5 }); // Clicks 5 times

Additional Click Options:

await locator.click({ button: 'right' });

This will **trigger the context menu**, but **Playwright cannot access it** since it is not part of the page's DOM.

Testing Checkbox and Text Area Interaction

Exercise

Write a test to check a checkbox that enables a text area, type a message into it, and verify the entered text. If we attempt to type while it is disabled, the test should fail.

Steps to Complete the Exercise:

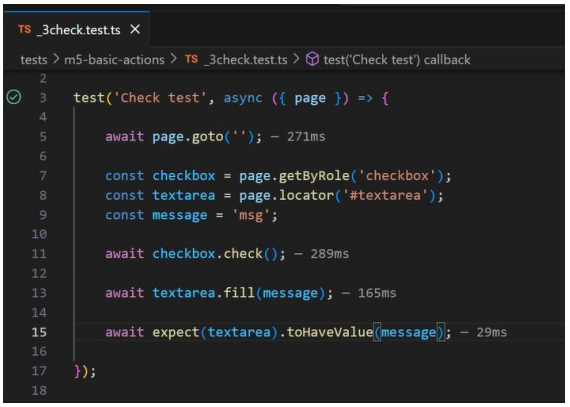
- Locate the Text Area.
- Attempt to Type While Disabled.
- Enable the Text Area.
- Type a Message into the Text Area.
- Verify the Entered Text.

The toContainText function is used for verifying text located between HTML tags. However, to verify the text entered into a text input field, the toHaveValue function must be used.

Testing a Checkbox That Enables a Text Area

Answer:

- **toContainText is used for verifying text inside HTML elements.**
- **toHaveValue should be used for verifying input field values.**
- **Running the test now should pass successfully.** ✅



Handling Dialogs in Playwright

- **Types of JavaScript Dialogs:**
  - Alert Dialog – Displays a message with a single OK button.
  - Confirm Dialog – Typically asks a question with OK and Cancel buttons.
  - Prompt Dialog – Expects user input with OK and Cancel buttons.

Dialog Behavior in Playwright:

•By default, **dialogs are dismissed automatically** unless a **special event listener** is added.

Key Takeaways:

•**Alerts, confirms, and prompts** can be intercepted using the page.on('dialog') event.

•Use **dialog.accept()** to confirm or provide input for prompts.

•Use **dialog.dismiss()** to cancel a confirm or dismiss a prompt.

Dialogs - Example

- **First test without dialog handling.**

```
tests > m6-advanced-actions > TS _1dialogs.test.ts > test('Dialog test - Default handling is to dismiss') callback
import { test, expect } from '@playwright/test';

const name = 'Sofia';

test('Dialog test - Default handling is to dismiss', async ({ page }) => {

    await page.goto('/');

    const input = page.getByLabel('First name');
    await input.fill(name);
    await expect(input).toHaveValue(name);

    await page.getByRole('button', { name: 'Clear' }).click();
    await expect(input).toHaveValue(name);

});
```

Second test with dialog interaction

```
test('Dialog test - OK or Dismiss', async ({ page }) => {

    page.on('dialog', dialog => dialog.accept());

    await page.goto('/');

    const input = page.getByLabel('First name');
    await input.fill(name);

    await page.getByRole('button', { name: 'Clear' }).click();
    await expect(input).toHaveValue('');

});
```

The first test is fully pre-coded to show that Playwright dismisses dialogs by default. Fill in one input, make sure it is filled in, then click Clear, and check that the input still has whatever typed in. Run this test, and it passes.

page.once('dialog', callback)This listens for the next occurrence of a dialog event (e.g., alert, confirm, prompt).The once method ensures the listener is triggered only once, meaning it won't persist for future dialogs.dialog => dialog.accept()

When a dialog appears, the provided callback function executes.The dialog.accept() method is called to automatically accept the dialog (just like clicking "OK" in a real browser popup).

File Downloads in Playwright

- **What to Verify in a Download Test?**
  - The download link works.
  - The downloaded file is correct.

Key Considerations:

•Downloaded files are **temporarily saved** and get **deleted** when the **browser context closes**.

•Some downloads (e.g., **PDFs opened in a viewer**) **cannot be clicked** by Playwright because the viewer is **not part of the HTML DOM**.

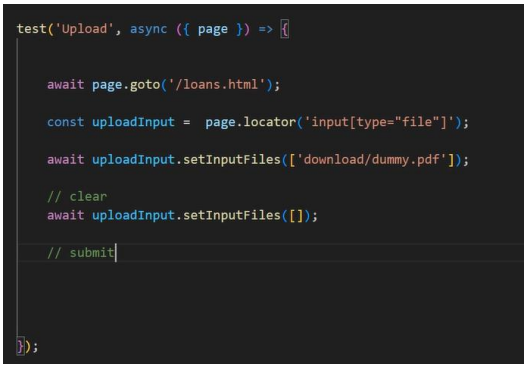
•Use **waitForEvent('download')** to handle file downloads.

•**Non-previewable files** can be downloaded in both **headed** and **headless** modes.

•**Previewable files (e.g., PDFs in a viewer)** can only be downloaded in **headless mode**.

File Uploads

- **Locate the file input element—typically an <input> of type file.**
- **Use the setInputFiles function to specify the file path, starting from the root of your project.**
  - Uploading multiple files: Enclose the file paths in square brackets to pass them as an array.
  - Clearing the input: Simply pass an empty array ([])



```
test('Upload', async ({ page }) => {

    await page.goto('/loans.html');

    const uploadInput =  page.locator('input[type="file"]');

    await uploadInput.setInputFiles(['download/dummy.pdf']);

    // clear
    await uploadInput.setInputFiles([]);

    // submit

});
```

- Identifies the input element, typically of type file.
- Uses the setInputFiles function to specify the file path, starting from the project root.
- Adds square brackets to pass multiple files as an array.
- Clears the input by passing an empty array.

Capturing Screenshots in Playwright for Debugging

- **Taking Screenshots with Playwright**
  - Capturing a basic screenshot:
    - Use page.screenshot() to take a screenshot. If no parameters are passed, it returns a promise of a buffer containing the image data.
  - Saving screenshots to a file:
    - Specify a file name and directory. If the directory doesn't exist, Playwright will create it automatically.
  - Capturing the full page:
    - Use the full-page option to capture the entire scrollable page instead of just the visible viewport.

As they say, *a picture is worth a thousand words.* Screenshots are invaluable for troubleshooting and root cause analysis. If a test fails after clicking a button, a screenshot can help you understand what went wrong.

Capturing Screenshots in Playwright for Debugging

- Saving screenshots in different formats:

  Specify formats like JPEG by including the file extension in the path.
- Masking sensitive data:
  - Use locators to find elements containing sensitive data.Apply .all() to hide all matching elements before taking a screenshot.
- Automating Screenshots on Test Failures
  - Enable automatic screenshots on failure:
  - Inside the global configuration file, set the use option:
    - use: { screenshot: 'only-on-failure' }
    - This applies to all tests without needing to write screenshot calls manually.

Mastering Playwright Test with Node.js

RUBRiC

- **In this lesson you will learn about:**
  - Playwright's Test Runner
  - Skipping and FailingTests
  - Grouping and Organizing Tests
  - Setup and Cleanup Hooks
  - Test Parameterization and Parallelization

92

Skipping and Failing Tests in Playwright

- **Why Skip a Test?**
- **There are multiple reasons why you might want to skip a test, such as:**
  - ✅ **The feature is not yet implemented**
  - ✅ **The test is not relevant for a specific browser or environment**
  - ✅ **The test relies on an unavailable precondition**

```
import { test, expect } from '@playwright/test';

test.skip('Will not run', async () => {

    console.log("This should not be printed");
});


test('Skip (un)conditionally', async ({ page, browserName }) => {

    test.skip(browserName === 'chromium', 'Does not work on Chromium, ticket ABC-123');

    test.skip(await page.getByTestId('someId').count() === 0, 'Skipping because at least 1 element X must be present')
});
```

Skipping tests

- **Mark a Test as "Fix Me" (fixme)**
  - Use test.fixme() to indicate that a test is currently broken

```
test.fixme('This test is broken and needs a fix', async ({ page }) => {
  // Test logic
});
```

  - It will be **marked as skipped** in the test report.

- **Handling Expected Failures (fail)**

1 The test.fail() annotation is useful for known bugs. It marks a test as expected to fail.

2 If the test actually fails, it counts as a pass (since failure was expected).

3 If the test unexpectedly passes, it counts as a failure.

```
test('This test should fail', async ({ page }) => {
  test.fail(); // Mark this test as expected to fail
  expect(2).toBe(3); // This assertion is false
});
```

File-Level Annotations in Playwright

- **Playwright allows you to apply annotations at the file level, meaning they will affect all tests in that file.**
- **This is useful when:**
  - The entire suite is irrelevant in a certain environment.
  - You want to temporarily disable an entire test file.
  - You want a consistent behavior across all tests in the file.

```
import { test, expect } from '@playwright/test';

test.skip('Will not run', async () => {

    console.log("This should not be printed");
});

test('Skip (un)conditionally', async ({ page, browserName }) => {

    test.skip(browserName === 'chromium', 'Does not work on Chromium, ticket ABC-123');

    test.skip(await page.getByTestId('someId').count() === 0, 'Skipping because at least 1 element X must be present')
});


test.fixme('Fixme', async () => {


});
```

Key Takeaways

✔ **File-level annotations override individual test annotations.**
✔ **skip and fixme do not conflict, as both result in skipped tests.**
✔ **skip overrides fail, meaning a test expected to fail will instead be skipped.**
✔ **Use file-level annotations when you want a consistent effect on all tests in a file.**

By leveraging these annotations, you can dynamically control test execution and better manage your Playwright test suite.

Grouping of tests in Playwright

- **As projects grow, they accumulate hundreds or even thousands of tests—not just unit tests, but also integration and end-to-end tests.**
- **Proper organization is essential for maintainability, readability, and debugging.**
- **Playwright allows tests to be grouped within a file using test.describe(). This method requires:**
  - A string – Defines the group title.
  - A callback function – Contains the related test functions.

```
import { test } from '@playwright/test';

test.describe('Feature A Group', () => {

    test('Test A.1', async ({ page }) => {
        await page.goto('');
        console.log("Test A.1");
    });


    test('Test A.2', async ({ page }) => {
        await page.goto('');
        console.log("Test A.2");
    });

});
```

Why Group Tests?

- Improves Organization: Groups related tests together.
- Enhances Readability: Makes the test structure easier to understand.
- Scopes Setup and Teardown: Allows shared setup within a group.
- Enables Selective Execution: Groups can be executed or skipped as needed.

Grouping Tests

- **Understanding Playwright's Test Execution Order**
- **Tests do not execute in the order they are written.**
- **Playwright randomizes test execution to promote independent test design.**
- **Best Practice: Each test should be self-contained, avoiding dependencies on other tests.**

Test Parameterization in Playwright

- **What is Test Parameterization?**
  Test parameterization allows the same test scenario to run with multiple input values, expecting consistent results. Instead of writing separate tests, a single test can handle multiple inputs efficiently.
- **Does Playwright Support Parameterization?**
  - The official documentation suggests using loops to achieve parameterization.

Implementing Parameterization in Playwright

- **Handling Duplicate Test Titles**
  - Playwright will throw an error if multiple tests have the same title.
  - Solution:
    - Use string interpolation (backticks) to include the parameter in the test title.
  - Unique titles ensure Playwright can **differentiate tests**.

```
import { test } from '@playwright/test';
const people = ['Alice', 'Bob'];

for (const name of people) {
    test(`Testing ${name}`, async () => {
        console.log(name);
    });
}
```

Handling Multiple Values with Arrays

- **If multiple parameters need to be tested together, an array can be used.**
- **When testing large datasets, storing values in code is not ideal.**
- **Best Practice: Store test data in external files (e.g., CSV, JSON).**

```
const inputs2 = [
    ["10", "6 months", "After 6 Months you will earn $0.20 on your deposit"],
    ["20", "1 Year", "After 1 Year you will earn $1.00 on your deposit"]
];

for(const [sum, period, result] of inputs2) {
    test(`testing with ${sum} ${period} ${result}`, async ({ page }) => {

        await page.goto('/savings.html');
        await page.getByTestId('deposit').fill(sum);
        await page.getByTestId('period').selectOption(period);

        await expect(page.getByTestId('result')).toHaveText(result);

    });
}
```

Understanding const, Arrays, and Iteration in JavaScript

•The const keyword **declares a variable (inputs2) that cannot be reassigned**.

•However, the contents of inputs2 **can be modified**, such as adding new values using push().

•inputs2 is an **array of nested arrays**, with each inner array representing test data.

Using for...of to Iterate Over inputs2

✅ for...of is used to iterate through each element in inputs2.
✅ Since inputs2 is an **array of arrays**, each element is itself an array.

Destructuring the Inner Arrays

Each nested array is destructured into three separate variables:

•**sum** → First element (e.g., "10", "20")

•**period** → Second element (e.g., "6 months", "1 Year")

•**result** → Third element (expected output message)

Playwright Parallelization: Understanding and Configuring Test Execution

✅ **Single Test File Execution**

- When running a single test file, Playwright **uses only one worker**, meaning **no parallel execution** within that file.
- By default, tests in a **single file** run **one after another (serially)**.

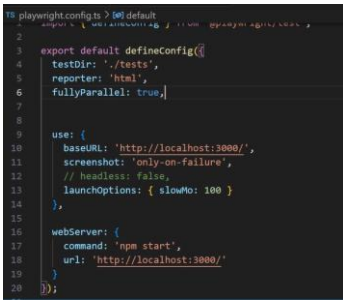
✅ **Multiple Test Files Execution**

- Playwright **runs multiple test files in parallel**, using **one worker per file**.
- This allows **better utilization of system resources** without modifying any configurations.

Note: To run all the tests in the test file: Run via command line -> npx playwright test <file name>

Enabling Parallel Execution Within a File

- **For a Group of Tests:**
  - Use test.describe.parallel() to run only that group in parallel.
- **For All Tests in a File:**
  - Add test.describe.configure({ mode: "parallel" }) at the top of the file.
  - This enables parallel execution within the entire test file.



Full Parallelization (Recommended Setting)

- **Global Parallel Execution**

•To enable **full parallelization across all test files and groups**, update the **global configuration:**

•fullyParallel: true in playwright config file under export `defineConfig`

•This ensures that tests **run independently, in parallel, and randomly**, maximizing efficiency.

Understanding Playwright's Default Worker Allocation

- **With fullyParallel mode enabled, Playwright dynamically allocates workers based on the number of tests and logical CPU cores.**
- **On a machine with 8 logical CPU cores, Playwright defaults to using 4 workers (50% of logical cores).**
- **The number of workers varies based on system specifications; less powerful machines may have only 1 or 2 workers.**
- **Overriding the Default Worker Count**
  - Users can override the default worker count via the command line or global configuration file.
  - Increasing the worker count does not always result in faster execution.
  - Excessive workers can lead to resource contention, reducing overall efficiency.

Finding the Optimal Worker Configuration

- **The Microsoft documentation provides insights into determining the best test suite configuration.**
- **Optimal configuration depends on the balance between available CPU resources and task load.**
- **Resource Contention and Performance Considerations**
  - More workers lead to competition for CPU resources, impacting performance.
  - Similar to having too many browser tabs open, excessive parallel execution slows down individual tasks.
  - Playwright's default of 1 worker per 2 logical processors is a generic but effective balance.

Note: **Advanced Optimization: Sharding**

•If local parallel execution is insufficient, Playwright supports sharding across multiple machines.

•Sharding requires knowledge of CI/CD infrastructure such as GitHub Actions.

•This technique enables large test suites to run efficiently across distributed environments.

Using the HTML report for Playwright

- **HTML reporter produces a self-contained folder that contains report for the test run that can be served as a web page.**
- **By default, HTML report is opened automatically if some of the tests failed.**
- **This behaviour can be controlled via the open property in the Playwright config.**
- **The possible values for that property are:** always**,** never **and** on-failure**(default).**
- **A quick way of opening the last test run report is**: npx playwright show-report

106

Using the command line to all tests and view the HTML report: **npx playwright test --reporter=html**

Example below:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  reporter: [['html', { open: 'never' }]],
});
```

Page Object Model (POM)

Page Object Model

RUBRiC

- **In this lesson you will learn about:**
  - What is POM
  - Advantages of using POM
  - Example of Using POM with Playwright Framework
  - Using POM with Cucumber and Playwright

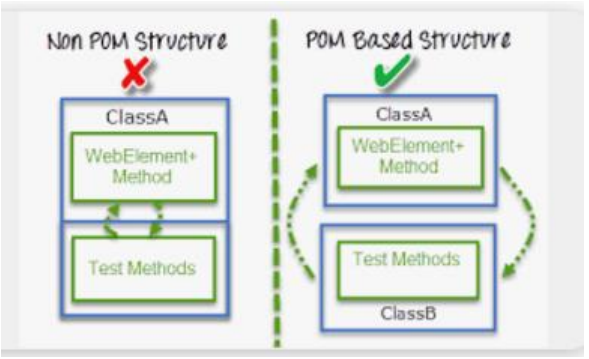
10

Page Object Model

- **Page Object Model is a design pattern** that stores web elements as a repository in a separate class file.
- Each class file contains separate web page elements.The web pages are separated into each class file as shards and automated.
- Page Object Model has nowadays become very popular test automation framework in the industry and many companies are using it because of its easy test maintenance and reduces duplication of code.

Page object models

- Large test suites can be structured to optimize ease of authoring and maintenance. Page object models are one such approach to structure your test suite.
- A page object represents a part of your web application.
- Example:
  - An e-commerce web application might have a home page, a listings page and a checkout page. Each of them can be represented by page object models.



Advantages of Page object models

- Easy Maintenance
  - Helps manage UI changes efficiently (e.g., drop-down changed to a radio button).
  - Identifies the correct page or screen to modify.Reduces errors and makes test cases easier to maintain.
- Code Reusability
  - Screens are independent, allowing test code reuse.No need to rewrite code, saving time and effort.
- Improved Readability & Reliability
  - Each screen has an independent TypeScript file.
  - Actions for a specific screen are easily identifiable.
  - Changes can be made efficiently without affecting other files.

Disadvantages of Using POM

- It takes time to build the framework initially.
- A good understanding of code is required for POM.
- A tiny miscalculation can destroy the whole test suite, as the elements are stored in a shared file.

Implementation of Page Object Model with Playwright

- **Create a folder called: pageObjects**

1 Purpose of LoginPage Class

- Encapsulates login functionality using the Page Object Model (POM).
- Improves code reusability and maintainability in Playwright tests.
- Provides easy-to-use methods for interacting with login elements.

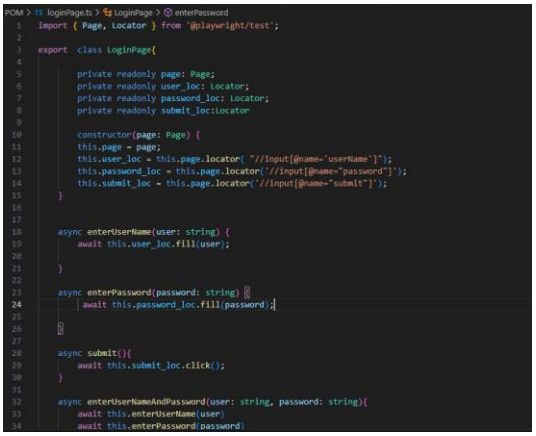
2 Class Structure

- Constructor: Initializes the Playwright Page and defines locators for login elements.
- Methods: Provides functions to interact with login elements.

Playwright Page Object Model: LoginPage Class

3 Code Breakdown

- **Instance Variables (Locators)**
  - user_loc: Locator for the username input field.
  - password_loc: Locator for the password input field.
  - submit_loc: Locator for the submit button.
- **Constructor (constructor(page: Page))**
  - Accepts a Page object.
  - Initializes locators using Playwright's locator() method.



Common Data & Login Data in Playwright

1 Purpose of CommonData

- Stores global configuration values used across multiple tests.
- Keeps URLs, API endpoints, and other static data separate from test logic.
- Helps in maintaining consistency and flexibility.

2 CommonData Structure

- **applicationUrl:**

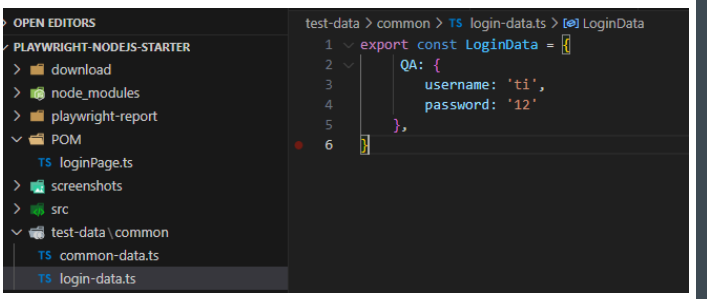
  Stores the base URL of the application.

  This URL can be reused across multiple test cases to avoid hardcoding it in every test.

```
export const CommonData = {
    applicationUrl: 'https://demo.guru99.com/test/newtours/'
};
```

Common Data & Login Data in Playwright - Implementation

- **Create a folder ‘test-data’**
- **Inside create a ts file ‘common-data.ts’**
  - Add the common data as an object by exporting it.
- **Create another ts file called ‘login-data.ts’**
  - Add the test data inside the file as an object by setting it as export.



Creating the test suite in Playwright

- **Imports Required Modules:**

```
import { test, Page, expect } from '@playwright/test';
import { LoginPage } from '../../POM/loginPage';
import { LoginData } from '../../test-data/common/login-data';
import { CommonData } from '../../test-data/common/common-data';
```

- **Defining Test Variables**

```
let page: Page;
let loginPage: LoginPage;
const loginData = LoginData.QA;
```

✅ **test, Page, expect** → Used for writing and executing tests.
✅ **LoginPage** → Page Object Model (POM) for handling login actions.
✅ **LoginData & CommonData** → External data sources for credentials & URLs.

✅ **page**: Manages browser interactions.
✅ **loginPage**: An instance of LoginPage for login operations.
✅ **loginData**: Retrieves credentials from LoginData.QA.

Implementing the test cases in Playwright with POM

- **test.describe.serial('Book a flight and logout', async () => {...});**
  - Defines a test suite named "Book a flight and logout".Runs tests serially (one after another) to avoid conflicts
- **Declares:**
  - let page: Page;
    - → Stores the Playwright page instance.
  - let loginPage: LoginPage;
    - → Stores an instance of LoginPage.
  - const loginData = LoginData.QA;
    - → Stores QA environment login data.

Adding the Hooks in the test file

- **beforeAll & afterAll Hooks**
- ✅ **beforeAll → Opens a new browser page & initializes LoginPage.**
- ✅ **afterAll → Closes the browser after tests finish.**

```
test.beforeAll(async ({ browser }) => {
   page = await browser.newPage();
   loginPage = new LoginPage(page);
});

test.afterAll(async () => {
   await page.close();
});
```

Login Test

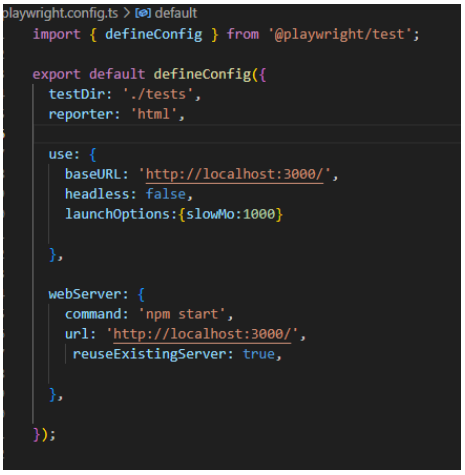
✅ Navigates to the login page.
✅ Fills username & password dynamically.
✅ Submits the form & verifies successful login.

```
test('Login', async () => {
    await test.step('Open browser and navigate to website', async () => {
        await page.goto(CommonData.applicationUrl);
    });

    await test.step('Enter credentials and log in', async () => {
        await loginPage.enterUserNameAndPassword(loginData.username, loginData.password);
        await loginPage.submit();
    });

    await test.step('Verify successful login with welcome message on home page', async () =>
        await expect(page.locator('h3')).toHaveText('Login Successfully');
    });
});
```

✅ Uses Page Object Model (POM) for reusability.
✅ Data-driven approach avoids hardcoding credentials.
✅ Modular test steps improve readability & debugging.
✅ Playwright's expect() ensures accurate validation.

Running tests

- **Using the Playwright for VS code extension, run the test.**
- **Run the command to view the sample report generated by Playwright:**
  - npx playwright show-report
- **Ensure that** `reporter: 'html',`
  Is present in the config file.



Implementation of Page object Model (POM) with BDD (Cucumber)

- **Create a folder and save it.**
- **Open the new folder in VS Code.**
- **Open a new terminal and run the command:**
  - npm init playwright @latest
- **Download cucumber by running:**
  - npm i @cucumber/cucumber –D
- **Run the command to install typescript and npm:**
  - npm i ts-node –D
- **Download the extension : Cucumber (Gherkin) Full Support by Alexander Krechik**

Note: Ensure no other cucumber extensions are installed, as it could create a conflict in the cucumber(settings.json)file. To troubleshoot, use ctrl and ,

Implementation of Page Object Model with BDD

- **Next, create a cucumber .json file and add the following contents:**

```
{
    "default": {
        "formatOptions": {
            "snippetInterface": "async-await"
        },
        "paths": ["src/test/features/"],
        "publishQuiet": true,
        "dryRun": false,
        "require": [
            "src/test/steps/*.ts"
        ],
        "requireModule": [
            "ts-node/register"
        ]
    }
}
```

Create a tsconfig.json file

- **Create a tsconfig.json file**
  - In this code:
  - 1. compilerOptions
    - This is the main object where TypeScript compiler options are defined.
  - "module": "CommonJS“
    - This specifies that TypeScript should compile the code using the CommonJS module system.
    - CommonJS is the module system used in Node.js (with require() and module.exports).
  - "target": "ES6“
    - This tells TypeScript to compile the code to **ECMAScript 6 (ES6)**.

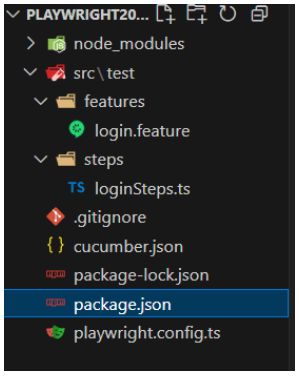
4. "esModuleInterop": true

•This allows **better interoperability between CommonJS and ES Modules**.

•When enabled, it allows using ES module-style import statements for CommonJS modules.

Implementation of Page Object Model with BDD

- **Modify the package.json file to add the following:**
- **Create the folder structure**



1. "scripts" section

•In **package.json**, the "scripts" section defines shortcut commands that can be run using npm run <script-name>.

2. "tests" key

•This defines a script named **tests**.

•It can be executed using:

- •npm run tests

Implementation of Page Object Model with BDD

- **Inside the feature file, write the test case:**

```
Feature: Login

    Scenario: Login with submit button
        Given user is on the login page

        When user submit the login form with login 'complet@sdworx.com' and password 'SDWorx2025!'
        Then user should be on the folder page
```

- **Run the command: npm run tests**

Playwright Page Fixture Setup

- **Importing Playwright Page Module**
  - Imports Page from @playwright/test.
  - Defines a page variable (let page: Page).
  - Creating pageFixture Object
    - Declares pageFixture as an exported constant.
    - Initializes page property with undefined (TypeScript ignored).
    - Allows global access to a Playwright page instance across tests.
  - Purpose of pageFixture
    - Provides a centralized page instance for test execution.
    - Ensures the same page instance is shared across different test files.
    - Avoids the need to pass the page object manually in each test.

src > test > Utils > TS pageFixture.ts > pageFixture

```
1  import { Page } from '@playwright/test'
2
3  let page:Page;
4
5
6  export const pageFixture = {
7      // @ts-ignore
8      page: undefined as Page,
9
10 }
```

- **Key Benefits**
- **Reusability** – Shared page instance across multiple test steps.
- **Simplified Test Code** – No need to create a new Page instance in every test.
- **Improved Maintainability** – Centralized page management for easier debugging.

Playwright & Cucumber Setup in TypeScript

- **Global Setup (BeforeAll Hook)**
  - Launches a Chromium browser (non-headless mode).
  - Creates a new browser context for test isolation.
  - Initializes a new page and assigns it to pageFixture.page.
- **Closing Resources (AfterAll Hook)**
  - Closes the page instance after all tests.
  - Closes the browser context to free up memory.

```typescript
import { chromium, Page, Browser, BrowserContext } from '@playwright/test'
import { BeforeAll, AfterAll, Before, After, Status } from '@cucumber/cucumber'
import { pageFixture } from './pageFixture';


let browser:Browser;
let context:BrowserContext;
let page:Page;

BeforeAll(async function(){

    browser = await chromium.launch({headless: false})
    context = await browser.newContext()
    page = await context.newPage()  //page is being initialised here
    pageFixture.page = await page;

})
```

```typescript
AfterAll(async function(){
    await pageFixture.page.close()
    await context.close()
})
```

Cucumber json file setup

- **What Does It Do?**
  - The "require" field in the configuration loads TypeScript files before executing tests.
  - The pattern "src/test/Utils/*.ts" ensures that all TypeScript files inside the Utils folder are automatically included in the test run.
- **Purpose of Utils/*.ts Files**
  - Typically contains helper functions, utilities, and reusable methods.
  - Helps keep test steps (steps/*.ts) clean and modular by offloading common actions.

- **Why Is It Important?**

✅ Ensures all required **utility files** are available before test execution.
✅ Promotes **code reusability** by centralizing shared functions.
✅ Improves **test maintainability** by separating logic from step definitions.

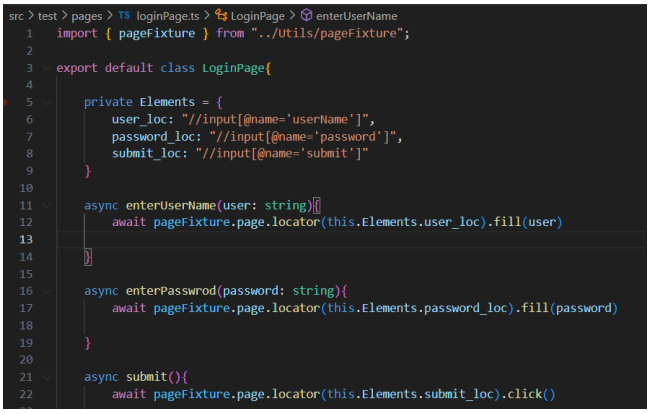
LoginPage class

- **Purpose of LoginPage**
- **ClassImplements Page Object Model (POM) to encapsulate login page actions.**
  - Improves code reusability and maintainability in Playwright tests.
    - Key Features

Encapsulates Locators:

- Stores element locators in a private object (Elements).
- Uses XPath to identify username, password, and submit button fields.

Modular Functions for Actions:enterUserName(user: string):



Step Definitions

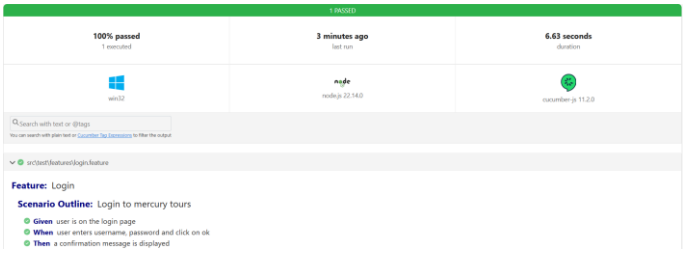
- **Purpose of This Code**
  - Implements Cucumber step definitions for login functionality in Playwright.
  - Uses Page Object Model (POM) for better structure and reusability.
- **Why Use Page Object Model (POM)?**
  - Reusability: LoginPage methods can be reused across different test cases.
  - Maintainability: If UI elements change, only LoginPage needs updating.
  - Readability: Test steps remain clear and human-readable.

```
import { Given, When, Then } from '@cucumber/cucumber';
import { pageFixture } from '../Utils/pageFixture';
import LoginPage from '../pages/loginPage';

const loginPage = new LoginPage()

Given('providing valid url for login',async function () {
    await pageFixture.page.goto("https://demo.guru99.com/test/newtours/")

 });

 When('providing valid username and password',async function () {
   await loginPage.enterUserNameAndPassword("mercury", "mercury")
 });

 Then('clicking login button',async function () {
   await loginPage.submit()
 });

 When('providing valid username as {string} and password as {string}',async function (name, password) {
   await loginPage.enterUserNameAndPassword(name, password)
 });
```

HTML Report with Cucumber

- **To generate an HTML report add the following code to cucumber.json file:**

```
{} cucumber.json ×   login.feature   tsconfig.json   loginStep
{} cucumber.json > {} default
  "default": {
    },
    "paths": ["src/test/features/"],
    "publishQuiet": true,
    "dryRun": false,
    "require": [
      "src/test/steps/*.ts"  ,
      "src/test/Utils/*.ts"
    ],
    "requireModule": [
      "ts-node/register"
    ],
    "format":[
      "html:test-result/cucumber-report.html",
      "json:test-result/cucumber-report.json",
      "rerun:@rerun.txt"
    ]
  }
}
```



Playwright API

Playwright API

- In this lesson you will learn about:
  - API Tests using Playwright
  - Navigate between pages
  - Capture Asynchronous data
  - Capture data after react re-renders
  - Managing Test Flows

RUBRiC

13
4

Introduction to API

- API testing is a type of software testing that analyses an application program interface (API) to verify it fulfils its expected functionality, security, performance and reliability.
- It also enables testers to make requests that might not be possible through the UI -- a necessity for exposing security flaws.
- API testing enables developers to access the app without a UI, helping the tester identify errors earlier in the development lifecycle -- rather than waiting for them to become bigger issues.

Accessing HTTP Layer in Playwright

Capturing and Manipulating HTTP Traffic

- **Pure API Testing**
- **Bypasses the UI entirely and interacts with the backend via HTTP requests.**
- **Common operations:**
  - **POST**: Create data.
  - **GET**: Retrieve and verify created data.
  - **DELETE**: Remove records.

136

Capturing and Manipulating HTTP Traffic

API-Assisted End-to-End Testing

- **Combines UI and API interactions for more efficient testing.**
- **Examples:**
  - Set authentication headers before navigation to simulate login.
  - Inject test data via API before interacting with the UI.

```
tests > m2-setup > api_01.test.ts > test('Request / Response Overview') callback
1  import { test, expect } from '@playwright/test';
2
3  test('Request / Response Overview', async ({ page }) => {
4
5      const response = await page.goto(''); — 613ms
6
7      if (response === null) return;
8
9
10     console.log(response.url());
11     console.log(response.status());
12     console.log(response.ok());
13     expect(response.ok()).toBeTruthy(); — 1ms
14
15     console.log(await response.allHeaders());
16     console.log(await response.headersArray());
17
18     console.log(await response.body());
19     console.log(await response.text());
20
21     const request = response.request();
22     console.log(await request.allHeaders());
23     console.log(request.method());       // GET
24 });
25
```

Retrieving Response Details:

API Response:

- The URL of the page and the status code can be extracted.
- Playwright includes an ok function, which returns true if the status code falls within the 200s range.
- Since checking for a 200-range status is common, Playwright provides this specialized function.
- To assert this, the toBeTruthy function can be used.
- Response headers can also be retrieved, but they are returned as an object rather than an array.
- If an array format is needed, the headersArray function can be used.

138

Handling Response Body:

API Response body

- **The body of the response can be obtained in multiple ways:**
  - The body function returns a promise of a buffer object, which is not immediately readable.
  - The text function returns a text representation of the payload, suitable for handling JSON, HTML, or other text-based responses.
  - If the response contains JSON, the json function can be used, but it will throw an error if the payload is not valid JSON.
  - Running the test confirms that:
    - The text function successfully retrieves the HTML body.
    - Using json on an HTML response results in a parsing error.

139

Mixed End-to-End and API Testing

Hybrid testing

- **This approach is useful for writing mixed end-to-end tests, where browsing and HTTP assertions happen simultaneously.**
- **However, Playwright can also be used purely for API testing without involving a browser window.**

- **Using the request Fixture for API Testing:**
  - The request fixture allows Playwright scripts to act as an HTTP client without opening a browser.
  - Unlike the page fixture, which creates a browser instance, the request fixture does not, making it faster.
  - With this fixture, HTTP requests such as GET, POST, or DELETE can be sent.

140

Mixed End-to-End and API Testing

Example: Using the github API

- **With this fixture, HTTP requests such as GET, POST, or DELETE can be sent.**
  - A GET request can be issued to an API (e.g., GitHub API), and the response can be processed similarly:
    - Extracting the status code, headers, and JSON body.
    - Since the response is expected to contain JSON, the json function works correctly without errors.
  - Running this test confirms that:
    - The response headers and payload are retrieved successfully.

```
28  test('Request / Response', async ({ request }) => {
29
30      const response = await request.get('https://api.github.com/');
31
32      console.log(response.ok());
33      console.log(response.status());
34
35      console.log(response.headersArray());
36      console.log(await response.json());
37  });
```

Performing a Test with Mixed UI and API Layers using GitHub

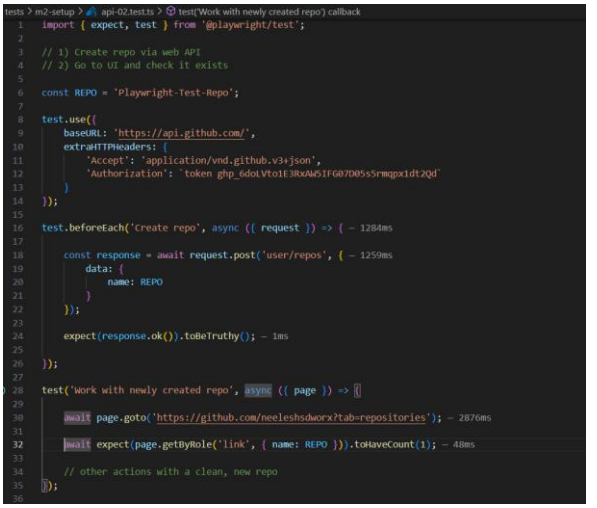
Test Scenario

- **The goal is to test actions around a freshly created GitHub repository.**
  - Instead of creating the repository via the UI, the REST API will be used for efficiency.
  - The test will involve:Setup: Creating a repository via the API.
  - Verification: Ensuring its existence through the UI.
  - Cleanup: Deleting the repository after the test.
- **Create a gitHub account.**
- **Use a temporary, personal token for authentication:**
  - Tokens should never be published in public repositories.
  - Generating a token:
    - Navigate to GitHub Profile → Settings → Developer settings → Personal access tokens.
    - Generate a new token with necessary permissions and delete it after use.

142

Setting Up the Repository (API Layer)

- **Define the repository name as a variable.**
- **Use test.use to set the base URL for the API to reduce duplication.**
- **Utilize the request fixture to send a POST request to the GitHub API endpoint for repository creation.**
- **Set up the required headers:**
  - Accept header (recommended but not mandatory).
  - Authorization header with a personal access token.



143

Verifying the Repository in the UI (UI Layer)

•Navigate to the GitHub profile page under the **Repositories** tab.

•Locate the newly created repository using appropriate **locators**.

•Assert that the repository exists and that only one such repository is present.

•Proceed with further actions (e.g., adding files) if needed.

Cleaning Up After the Test

•Send a DELETE request to the correct GitHub API endpoint to remove the repository.

•Authentication is required to avoid rejection.

•Validate cleanup success by checking the response status code:

- A successful deletion returns **204 No Content**.

Code Optimization & Best Practices

•Refactoring Headers:

- Move repeated headers to a central location using test.use with extraHTTPHeaders.
- This reduces duplication in setup and cleanup.

Importance of API integration calls for Web tests using Playwright

- Quicker Release. GUI (Graphical User Interface) tests have a reputation for taking longer to release products.
- Better Test Coverage.
- Ease to Shift Left.
- Lower Maintenance to Do.
- Faster Bug Fixes.
- Reduced Testing Costs.

Assertions are important and very useful in any software automation tools to assert something during your test execution.

Playwright Tools

Playwright Trace Viewer

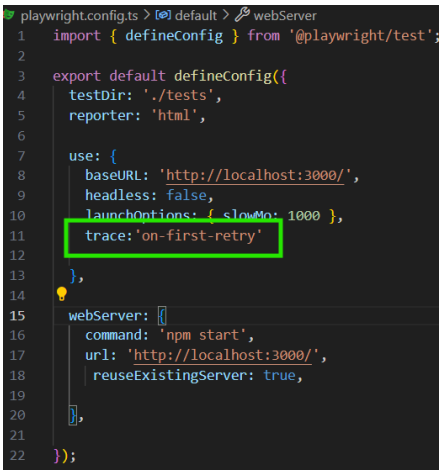
- **What is a Trace?**
  - Captures everything that happened during a test.
  - Includes actions performed, screenshots, logs, console messages, HTTP traffic, and metadata.
  - Allows users to replay and analyze failed tests.
- **Trace Configuration in Playwright**
  - Set in the global config under the use property.
  - Suggested setting: on-first-retry → Records a trace only if a test fails and is retried.
  - Tracing can be enabled via command line using --trace on.
- **Viewing the Trace in the Report**
  - If a test fails, the error section provides initial troubleshooting information.
  - The Trace section contains a downloadable .zip file for further investigation.

146

Other options exist, such as on-all-retries, but it may generate large amounts of data.

Exploring the Trace Viewer

- **Clicking on the trace screenshot opens the Trace Viewer in a separate page.**
- **The URL points to the trace .zip file.**
- **Timeline View at the top shows test execution step-by-step.Step-by-Step Analysis in the left pane allows reviewing actions and page state changes.**
- **Before and After Snapshots help observe modifications in the DOM.**
- **Bottom Pane provides:**
  - Playwright's internal logs
  - Captured errors
  - Console messages
  - HTTP traffic

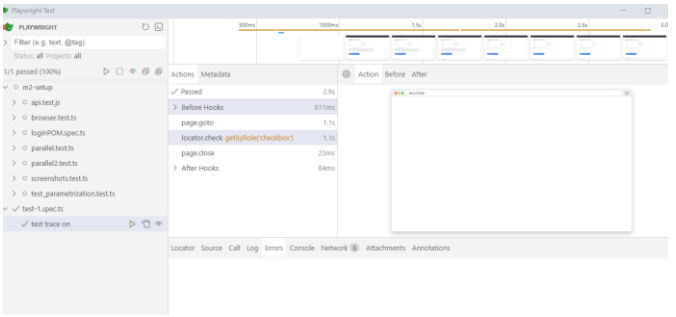


147

To run a test with trace viewer, type: npx playwright test <test file name> --trace on

UI Mode

- **What is UI Mode?**
  - Enables creating, running, and modifying tests within the Trace Viewer window.
  - Provides an intuitive visual interface for managing tests.
- **How to Enable UI Mode?**
  - Run the command: npx playwright test --ui
  - Does not execute tests immediately but opens a new window.
  - Displays the Test Explorer and the Trace Viewer side by side.



148

Benefits of UI Mode

•Allows running tests while **viewing their execution in Trace Viewer**.

•Enables real-time **test modifications and re-execution**.

•Enhances debugging with **better visual support**.

Case study

- **User Story: Automating a Form Submission on DemoQA**
  - As a user,I want to fill out and submit the "Practice Form" on DemoQA,So that I can validate successful form submission.

- **Acceptance Criteria:**
  - **Scenario: Successful Form Submission**
  - **Given** the user is on the "Practice Form" page at https://demoqa.com/automation-practice-form
    When the user fills in the form with valid details:
  - Enters **First Name**, **Last Name**, **Email**
  - Selects **Gender,** Enters a **Mobile Number**
  - Selects **Date of Birth,** Chooses a **Subject**
  - Selects a **Hobby,** Uploads a **Picture**
  - Enters the **Current Address**
    And clicks the **Submit** button
    Then the form should be submitted successfully
    And a confirmation message should be displayed
  - Name, gender and Mobile are mandatory fields and cannot be left blank.

Case study (Cont'd)

- As the Software Developer in Test, you are required to thoroughly test the user story as described, ensuring coverage of both positive and negative scenarios.
- Additionally, you are expected to automate these scenarios following best practices, including the use of Page Object Model (POM) for maintainability and reusability.
- Note: You will need to create a new playwright project for this exercise.

End

Well done!

151